

Sistemi di Calcolo

Cristian Amendola, Giovanni Esposito, Danilo Di Primio

Università degli studi La Sapienza

Ingegneria Informatica e Automatica - 2 anno

Febbraio - Maggio 2026

Indice

1	Debugging	3
1.1	GDB	3
1.2	Valgrind	4
2	Teoria	5
2.1	Cos'è un sistema di Calcolo?	5
	Hardware	5
	Elementi di base di un microprocessore	5
2.2	Come viene programmato un Sistema di Calcolo	8
	Linguaggio Alto e Basso Livello	8
	Compilatori e Interpreti	8
	Formato testo, oggetto ed eseguibile	9
	Disassemblare programmi in codice macchina: objdump	10
	Traduzione in linguaggio macchina programma C	10
3	Assembly	13
3.1	Funzioni	13
	Uso di parametri nelle funzioni	13
3.2	Istruzioni	13
	Struttura dei dati in memoria	14
	Accesso in memoria con spiazzamento	15
3.3	Gestione del flusso di esecuzione	16
	Salto incondizionato	16
	Salto condizionato	16
	Registro EFLAGS	17
3.4	Istruzioni di Condizione	18
	Cicli While-For	18
3.5	Puntatori	19
	Accesso in memoria con base,indice e spiazzamento	21
3.6	Chiamate a Funzione	22
	Metodo <i>Push + Pop</i>	26
3.7	Conversione del tipo (cast)	29
	Variabili Locali	31
	Espressioni Booleane	34
3.8	Allineamento nello Stack	35
3.9	Allineamento delle Struct	36
3.10	Istruzioni SHIFT	37
3.11	Istruzione IDIV	38
3.12	Istruzione CMOV	38
3.13	Ottimizzazione dei programmi	39
	Analisi prestazionale del programma	39
	Legge di Amdahl	40
3.14	Gestione dinamica della memoria in Assembly con <code>malloc</code>	40
	Gestione interna in assembly	41
3.15	Profilazione delle prestazioni	43
	Profilazione Manuale	43
	Profilazione Automatica	44
	Assetti chiave della ottimizzazione	44
	Tecniche di ottimizzazione del codice	45

3.16 Come vengono eseguiti i programmi	47
Terminazione dei programmi	48
Gestire la terminazione di un programma	48
Tipologie di Errore	49

Capitolo 1

Debugging

1.1 GDB

Uno dei requisiti necessari per utilizzare gli strumenti di debbuging è l'inclusione delle informazioni di debbuging durante la fase di compilazione. Per compilare un programma C con le informazioni di debug, bisogna compilare in questo modo:

```
gcc -g programma.c -o programma
```

In modo analogo per includere del codice assembly da noi scritto, bisogna aggiungere (Esempio caso codice assembly IA32) nella riga di compilazione:

```
gcc -g -m32 programma.c -o programma
```

Potrebbe capitare che il debug non funzioni, in questo caso è consigliato di definire in gdb i comandi custom, per farlo bisogna editare il file `.gdbinit` all'interno della home utente (Esempio sulla VM BIAR):

Andare nella directory: `/home/studente/.gdbinit`

Inserire nel file il seguente contenuto:

```
define go
    start
    layout src
    layout regs
    focus cmd
end
```

Il comando `go` ci permette di visualizzare diverse informazioni durante l'uso di GDB. Per avviare il debugging su un eseguibile "e" (dopo aver compilato): `→ gdb ./e`

Comandi principali di GDB:

- **go** → lancia il debug.
- **run** → rilancia l'esecuzione con eventuali aggiornamenti da specificare dopo (`run [argomento]`).
- **quit** → per uscire da gdb.
- **cont** → riprende l'esecuzione normalmente dopo un breakpoint.
- **step** → esegue una singola istruzione (non si ferma alle assegnazioni)
- **next** → esegue la funzione fino al return "finale" .
- **finish** → esegue fino alla fine della funzione in cui siamo, non funziona correttamente in codice assembly.
- **break** → inserisce un breakpoint, esempio `b e.c:20` inserisce un breakpoint alla linea 20 del file "e.c", per avere informazioni sui breakpoint attivi invece dovremmo usare `info break`, inoltre si può posizionare un break condizionale: `break e.c:20 if x == 0`, quindi in questo caso sarà posizionato il break solamente se x sarà uguale a 0, altrimenti continuerà normalmente.
- **print:**
 - **print \$eax** → stampa il contenuto di un registro.

- **print x** → stampa il contenuto della variabile x.
- **x addr** → stampa il contenuto della memoria all'indirizzo addr.
- **x\$eax + 4** → stampa il contenuto della memoria dato dall'espressione (in questo caso + 4 posizioni).
- **x/nfu addr** → stampa il contenuto della memoria all'indirizzo addr.
- **start** → si posiziona all'inizio del main, non c'è bisogno di utilizzarlo quando si usa go.
- **where** → mostra la posizione corrente nell'esecuzione.
- **list** → mostra il contenuto del file sorgente che contiene la funzione corrente.
- **backtrace** → ti dà l'elenco dei frame attivi.
- **refresh** → esegue il refresh dell'output del debugger.

1.2 Valgrind

Per utilizzare Valgrind avremo bisogno di compilare il nostro file allo stesso modo di gdb. Viene utilizzato per identificare molti errori in C, come:

- Accessi invalidi alla memoria (segmentation fault)
- uso di variabili non inizializzate
- mancata de-allocazione di blocchi allocati dinamicamente (memory leak)

La riga di codice per avviare Valgrind è la seguente:

```
valgrind --leak-check=full ./programma
```

Per terminare la esecuzione della macchina virtuale stando su VS:

```
sudo shutdown -h now
```

Capitolo 2

Teoria

2.1 Cos'è un sistema di Calcolo?

Un **sistema di calcolo** consiste di software e hardware che operano insieme per supportare l'esecuzione di programmi. In generale un sistema di calcolo è un qualsiasi **sistema programmabile**. Un sistema di calcolo può essere formato da un **singolo nodo**, cioè insieme di parti hardware strettamente connesse tra loro, oppure da **più nodi** connessi mediante una rete di comunicazione e viene anche chiamato **cluster**.

Hardware

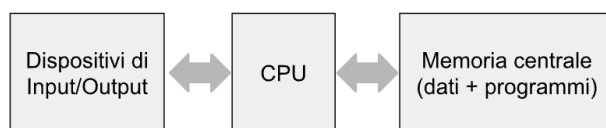


Figura 2.1: Schema di Von Neumann

La memoria centrale contiene dati da elaborare e programmi, i dispositivi di input e output scambiano dati e interagiscono con il mondo esterno, mentre la CPU esegue le istruzioni di programma (Modello di Von Neumann).

La **CPU** è il cuore del Sistema. Esso esegue programmi scritti in linguaggio macchina (codice nativo), rappresentati come sequenze di byte che codificano istruzioni.

I **Bus** sono strutture di interconnessione che collegano le varie componenti del sistema consentendo lo scambio dei dati. I bus sono canali di comunicazione che trasferiscono i dati a blocchi fissi di byte, chiamati word, che è generalmente di 4 byte oppure 8 byte.

L' **I/O Bridge** si occupa di coordinare lo scambio dati fra la CPU e il resto del sistema.

I **Controller e adattatori** interfacciano il sistema verso il mondo esterno, ad esempio acquisendo i movimenti del mouse o i tasti premuti sulla tastiera. La differenza fra i due è che i controller sono saldati sulla scheda madre oppure sono integrati nel dispositivo esterno, mentre gli adattatori sono schede esterne collegate alla scheda madre.

Il **Sistemi di memoria** di un calcolatore è molto articolato. La CPU non comunica direttamente con la memoria principale (RAM), ma ci sono una serie di strati di memorie intermedie chiamate cache che memorizzano i dati più frequentemente acceduti o adiacenti. Le cache hanno tempo di accesso molto inferiori a quelli di una RAM.

Elementi di base di un microprocessore

Negli elementi di base di un processore abbiamo le porte logiche che realizzano funzionalità elementari di una CPU. Le tre porte logiche principali sono: AND, OR, NOT e sono rappresentate dai seguenti simboli:

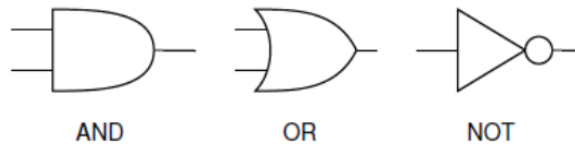


Figura 2.2: Porte Logiche

Analizziamo una porta alla volta, cominciando dalla porta **AND**, realizza un operatore binario che vale 1 se e solo se entrambi i propri argomenti sono 1:

X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1

Figura 2.3: Tabella Porta Logica AND

La porta **OR** realizza un operatore binario che vale 1 se e solo se almeno uno dei propri argomenti è 1:

X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1

Figura 2.4: Tabella Porta Logica OR

La porta **NOT** realizza un operatore unitario che vale 1 se e solo se il proprio argomento è 0:

X	NOT X
0	1
1	0

Figura 2.5: Tabella Porta Logica NOT

Abbiamo anche la possibilità di circuiti combinatori che realizzano funzioni booleane arbitrarie, vediamo un esempio di addizionatore parziale:

X	Y	somma(X,Y)	riporto(X,Y)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figura 2.6: Operazioni Logiche

Inoltre un'espressione, gli operatori hanno priorità, dalla maggiore alla minore in questo ordine: NOT, AND, OR.

2.2 Come viene programmato un Sistema di Calcolo

Linguaggio Alto e Basso Livello

Nei linguaggi di programmazione possiamo differenziarli in: alto livello e basso livello. Un linguaggio di **alto livello** forniscono costrutti molto più potenti e sono molto più semplici da imparare, esempi sono: C, Python, ecc... Programmatori più avanzati prediligono l'uso di linguaggi di **livello più basso** per avere un codice più ottimizzato per le prestazioni, esempio è il linguaggio Assembly.

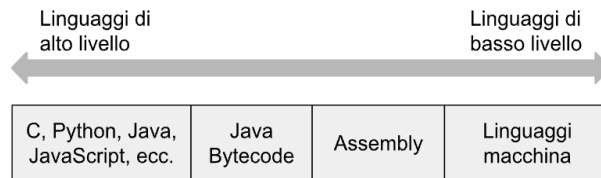


Figura 2.7: Classifica Linguaggi

Il linguaggio macchina è un linguaggio particolare progettato con l'obiettivo di essere facilmente interpretabile dai sistemi elettronici.

Compilatori e Interpreti

C'è da fare ancora una differenziazione tra i linguaggi, di fatto abbiamo due tipi di linguaggi: compilati e interpretati. Nei linguaggi **compilati** il programma di alto livello viene tradotto in codice macchina in modo da essere eseguito direttamente dalla CPU, questa operazione viene chiamata compilazione; mentre un linguaggio **interpretato** esegue direttamente il programma senza bisogno di compilarlo prima, questi linguaggi vengono chiamati linguaggi di scripting. Esiste anche la possibilità di avere un linguaggio **ibrido**, ovvero il programma viene prima compilato in codice scritto in un linguaggio intermedio per poi venire interpretato. Ogni metodologia comporta i vantaggi e svantaggi:

Compilati:

- Vantaggi → il compilatore genera un codice ottimizzato per la piattaforma di calcolo considerata ottenendo migliori prestazioni.
- Svantaggi → il programma non è portatile, è quindi eseguibile solo sulla piattaforma di calcolo su cui è stata compilata.

Interpretati:

- Vantaggi → il programma è portatile, inoltre è possibile eseguirlo direttamente senza bisogno di compilarlo.
- Svantaggi → l'esecuzione è più lenta, tuttavia alcuni interpreti, come Java Bytecode, utilizzano una tecnica chiamata Just-in-time compilation (JIT) che consiste nel compilare il programma da eseguire in codice nativo subito prima di essere eseguito.

Ibridi:

- Vantaggi → programma portatile.
- Svantaggi → stessi dei linguaggi interpretati.

Noi ci occuperemo del linguaggio C. La compilazione di un programma C può essere scomposta nella compilazione di più moduli C, detti translation unit, ciascuno con estensione ".c".

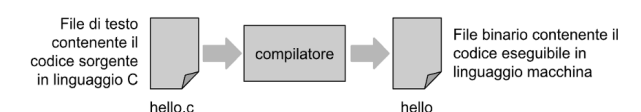


Figura 2.8: Esempio con C

Il processo di compilazione in C è composto in realtà da diversi stadi che vengono effettuati dal sistema di compilazione (compilation toolchain), senza rendercene conto:

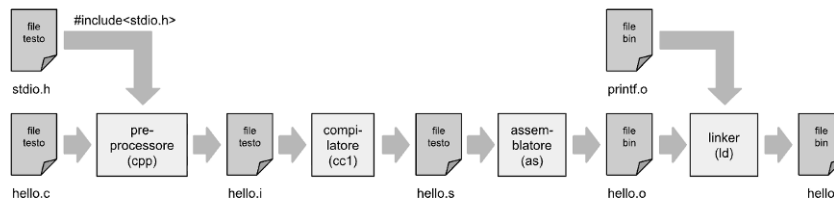


Figura 2.9: Esempio con C

Gli stadi coinvolgono diversi sottoprogrammi che vengono attivati nel sistema di compilazione:

- **Processore** → prende un testo C e lo converte in un altro testo C con le direttive già elaborate (`#include`, `#define`, ecc...). Quindi oltre che leggere il file “hello.c” leggerà anche il file della libreria “stdio.h”. Per intenderci, il comando: `[gcc -E hello.c]`.
- **Processore** → prende il testo C senza direttive, verifica la correttezza sintattica e lo traduce in assembly. Con il comando: `[gcc -S hello.i]`.
- **Processore** → prende un programma in assembly e lo traduce in codice macchina generando un file oggetto binario. Tramite: `[gcc -c hello.s]`
- **Processore** → prende i vari file oggetto e li collega insieme a formare un unico file eseguibile. Nel nostro caso collega insieme “hello.o” contenente il codice macchina e “printf.o” contenente il codice macchina che realizza la funzione “printf” invocata dal programma, dando come risultato il file eseguibile “hello”. Con l'utilizzo del comando: `[gcc hello.o -o hello]`.

Il collegamento può avvenire in due modi possibili: **linking statico**, nel quale il codice oggetto della funzione chiamata viene copiato nel file eseguibile, e **linking dinamico** dove il codice oggetto della funzione chiamata risiede in un file fornito con il sistema operativo, successivamente il collegamento tra la chiamata a funzione ed il codice che lo implementa viene fatto al caricamento del file eseguibile. Esisto in realtà un altro tipo di collegamento di linking chiamato **linking a Runtime**, che avviene durante l'esecuzione del programma, il collegamento tra programma e le librerie necessarie viene effettuato solo nel momento in cui serve realmente quella funzione, mentre il programma è già in funzione, permette di caricare le librerie solo quando necessario. Nei programmi possiamo riscontrare tipi diversi di errore, come gli **errori in fase di esecuzione** (Runtime) che rappresentano problemi che si verificano mentre il programma è in funzione, spesso dovuto all'incompatibilità tra parte software (programma) e hardware oppure a operazioni non valide. Esempio è la divisione per 0.

Abbiamo inoltre anche gli **errori in fase di compilazione** che si verificano durante la fase di compilazione, causati da errori di sintassi o di scrittura del codice, come parentesi mancanti, variabili non dichiarati ecc., e impediscono al programma di avviarsi finché non vengono corretti.

Instruction set architecture (ISA) ed è lo standard che definisce come funziona una struttura di calcolo, in modo più preciso un processore. All'interno del documento possiamo trovare:

- *Stato della CPU*
 - Registri → quanti e quali.
 - Memoria
- *Istruzioni*
 - Formato
 - Tipologie → spostamento dati, aritmetico/logiche, controllo.
- *Semantica delle istruzioni* → come l'esecuzione di ciascuna istruzione modifica lo stato.

Formato testo, oggetto ed eseguibile

Il formato binario ELF (Executable and Linkable Format) è un tipo di formato usato nei sistemi operativi per i file eseguibili, le librerie e i file oggetto. Contiene diverse parti tra cui:

- Informazioni sul programma
- Codice da eseguire
- Dati del programma

- Informazioni per il collegamento con librerie esterne

Non può essere eseguito o linkato su MacOS X. Allo stesso modo un file formato Mach-O non può essere eseguito o linkato in Linux; pertanto, i formati eseguibili non garantiscono la portabilità dei programmi.

Disassemblare programmi in codice macchina: objdump

Disassemblare un programma in linguaggio macchina consiste nel tradurne le istruzioni nel codice assembly corrispondente in modo che possano essere analizzate da un programmatore.

Linux: objdump -d (disassemblato)

In ambiente Linux è possibile disassemblare un file oggetto o un file eseguibile usando il comando `objdump -d`.

Consideriamo il seguente semplice modulo `somma.c`:

```
int somma(int x, int y) {
    return x + y;
}
```

Compilandolo otteniamo un file oggetto `somma.o`:

```
$ gcc -c somma.c
```

che possiamo disassemblare come segue:

```
$ objdump -d somma.o
```

Il comando `objdump` invia l'output sul canale standard (di default è il terminale):

Figura 2.10: Esempio Objdump

Traduzione in linguaggio macchina programma C

I sistemi di calcolo si basano su un certo numero di astrazioni che forniscono una visione più semplice del funzionamento della macchina. Due delle più importanti astrazioni sono:

- **Memoria** → vista come un grosso array di byte
- **ISA** (Instruction Set Architecture) → definisce:
 - Stato della CPU
 - Formato delle sue istruzioni
 - Effetto che le istruzioni hanno sullo stato

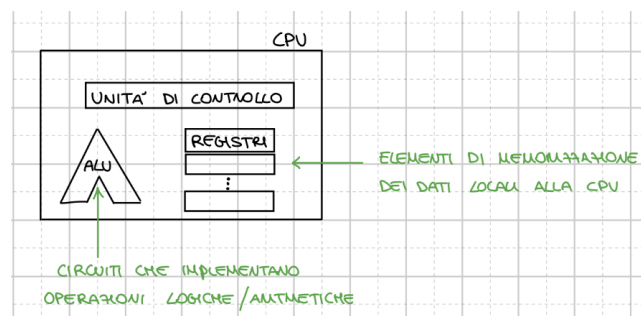


Figura 2.11: ALU

Per tradurre codice di alto livello in codice macchina, i compilatori si basano sulla descrizione astratta della macchina data dalla sua ISA. Le più diffuse sono:

- **IA32** → - IA32 → A, B, C, D, DI, SI, BP, SP, EP, IP, EFLAGS, i primi 7 sono general purpose, ovvero possono essere utilizzati a piacimento seguendo comunque le regole. **BP** e **SP** fanno parte della gestione dello stack. Abbiamo poi il registro **IP** (Instruction Pointer) è un registro non modificabile ma solo leggibile che viene gestito in maniera automatica dalla CPU, inserendo l'indirizzo della prossima istruzione da eseguire in memoria. Infine, abbiamo **EFLAGS**, anch'esso non modificabile che contiene dei flag che descrivono lo stato di funzionamento della CPU. Ciascun registro contiene 32 bit, così distribuiti:

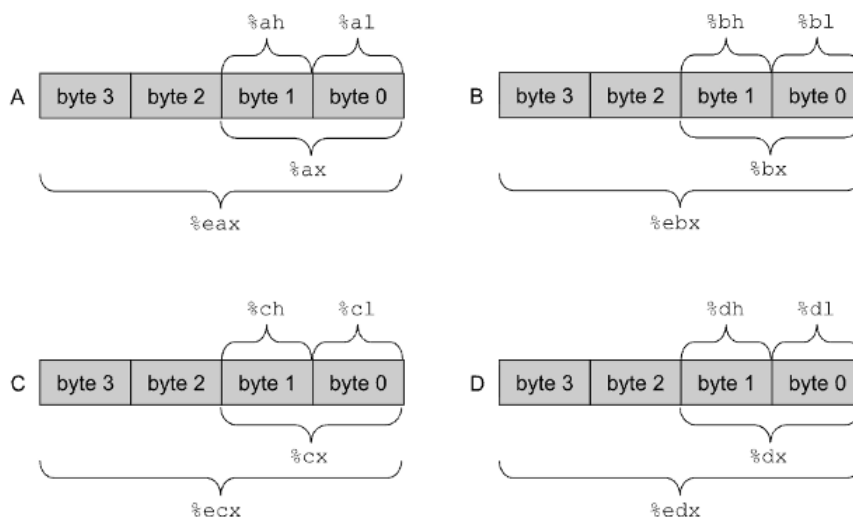


Figura 2.12: Registri IA32

Mentre gli altri registri hanno una suddivisione leggermente diversa:

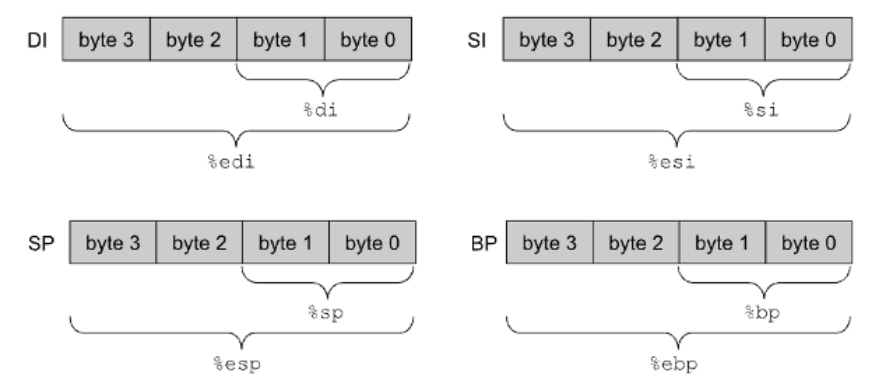


Figura 2.13: Registri IA32

I tipi macchina permettono di rappresentare sia numeri interi che numeri con virgola mobile. Vediamo in cosa corrispondono in C:

Tipo di dato C	Tipo di dato macchina	Suffisso	Dimensione in byte
char	Byte	b	1
short	Word	w	2
int	Double word	l	4
long	Double word	l	4
long long	<i>non supportato</i>	-	8
puntatore	Double word	l	4
float	Single precision	s	4
double	Double precision	l	8
long double	Extended precision	t	12 (10 usati)

Figura 2.14: Tipi di Dati

- **X86-64** → descrive le architetture della famiglia di processori x86 a 64 bit, è retrocompatibile con IA32, ovvero le istruzioni di IA32 sono presenti anche nell'x86-64.

Si hanno 11 possibili forme di operandi. Per gli operandi di memoria, ci sono vari modi di indirizzamento che consentono di accedere alla memoria dopo averne calcolato un indirizzo:

Tipo	Sintassi	Valore Denotato	Nome Convenzionale
<i>Immediato</i>	$\$imm$	imm	immediato
<i>Registro</i>	E	$R[E]$	Registro
<i>Memoria</i>	imm	$M[imm]$	Assoluto
<i>Memoria</i>	E_{base}	$M[R[E_{base}]]$	Indiretto
<i>Memoria</i>	$imm(E_{base})$	$M[imm + R[E_{base}]]$	Base e spiazzamento
<i>Memoria</i>	(E_{base}, E_{indice})	$M[imm + R[E_{base}] + R[E_{indice}]]$	Base e indice
<i>Memoria</i>	$imm(E_{base}, E_{indice})$	$M[imm + R[E_{base}] + R[E_{indice}]]$	Base, indice e spiazzamento
<i>Memoria</i>	(E_{indice}, s)	$M[R[E_{indice}] \cdot s]$	Indice e scala
<i>Memoria</i>	$imm(E_{indice}, s)$	$M[imm + R[E_{indice}] \cdot s]$	Base, indice, scala e spiazzamento
<i>Memoria</i>	$imm(E_{base}, E_{indice}, s)$	$M[R[E_{base}] + R[E_{indice}] \cdot s]$	Base, indice e scala
<i>Memoria</i>	$imm(E_{base}, E_{indice}, s)$	$M[imm + R[E_{base}] + R[E_{indice}] \cdot s]$	Base, indice, scala e spiazzamento

Tabella 2.1: Operandi Possibili

Abbiamo anche due modi in cui un processore definisce l'ordine in cui vengono disposti in memoria i byte della rappresentazione di un valore numerico (binari):

- **Big-Endian** → il bit più significativo viene posto all'indirizzo più basso.
- **Little-Endian** → il bit meno significativo viene posto all'indirizzo più basso.

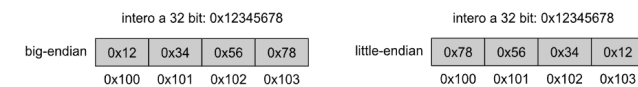


Figura 2.15: Big e Little Endian

L'architettura di IA32 è Little-Endian, mentre sono Big-Endian PowerPC e SPARC. Noi per il momento utilizzeremo i registri A, C, D.

Capitolo 3

Assembly

3.1 Funzioni

Per definire una funzione utilizziamo una "etichetta"/"label". Una funzione si definisce nel seguente modo →
nome_etichetta
esempio funzione somma:

```
SOMMA:
```

Per rendere una funzione visibile da altri file oggetto usiamo la direttiva ".globl". esempio

```
.globl SOMMA  
SOMMA:
```

esempio prima parte, codice C → Assembly

```
int f(){  
    int x;  
    x = 10;  
    return x;  
}  
→  
.globl f  
f:  
#x -> eax //prende tutto il registro
```

Uso di parametri nelle funzioni

```
int f(int x){  
    return x+1;  
}  
→  
int f(int x){  
    int a;  
    a=x;  
    a++;  
    return a;  
}  
→  
.globl f  
f:  
#a->eax  
movl ,%eax  
inc %eax  
ret  
f.c  
f_eq.c  
f.s
```

3.2 Istruzioni

Per ogni istruzione dobbiamo definire la taglia dei dati su cui andrà a operare seguendo la sua semantica. In IA32 abbiamo tre tagli possibili:

Esempio	Suffisso Assembly	Dimensione in byte
Char	b	1
Short	w	2
int, T*	l	4

Istruzioni per lo spostamento

S viene spostato in D

```
mov S, D
```

le istruzioni hanno in generale un operando sorgente che specifica un valore di ingresso per l'operazione e un operando destinazione che identifica dove deve essere immagazzinato il risultato

- Operando Sorgente
 - Immediato → operando immagazzinato insieme all'istruzione stessa dichiarato con \$, esempio \$10
 - Registro → operando memorizzato in uno degli 8 registri interni dichiarato con %, esempio %eax
 - Memoria → operando memorizzato in memoria
- Operando Destinazione
 - Registro → il risultato dell'operazione viene memorizzato in uno degli 8 registri interni, sempre dichiarato con %
 - Memoria → il risultato dell'operazione viene memorizzato in memoria

Regola: non si usano mai valori immediati per l'operando D

Regola: non è possibile usare l'indirizzamento a memoria per entrambi gli operandi

esempio seconda parte, codice C → Assembly

<pre>int f(){ int x; // dichiarazione x = 10; // assegnazione return x; } f.c</pre>	→	<pre>.globl f f: #x -> eax //prende tutto il registro movl \$10, %eax ret f.s</pre>
--	---	---

esempio con char codice C → Assembly

<pre>char f(){ char p; // dichiarazione p = 'a'; // assegnazione printf("%c\n",p) return 0; } f.c</pre>	→	<pre>.globl f f: #x -> al //prende tutto il registro movb \$0x61, %al ret f.s</pre>
--	---	---

Struttura dei dati in memoria

Per la struttura dei dati in memoria vediamo nello specifico il funzionamento dello StackPointer:

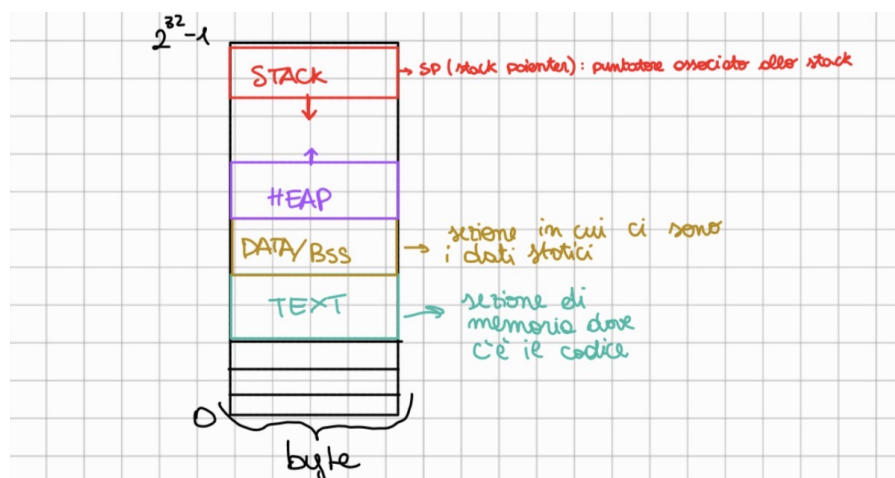


Figura 3.1: StackPointer

Vediamo però cosa accade di preciso nella memoria quando viene chiamata la funzione f:

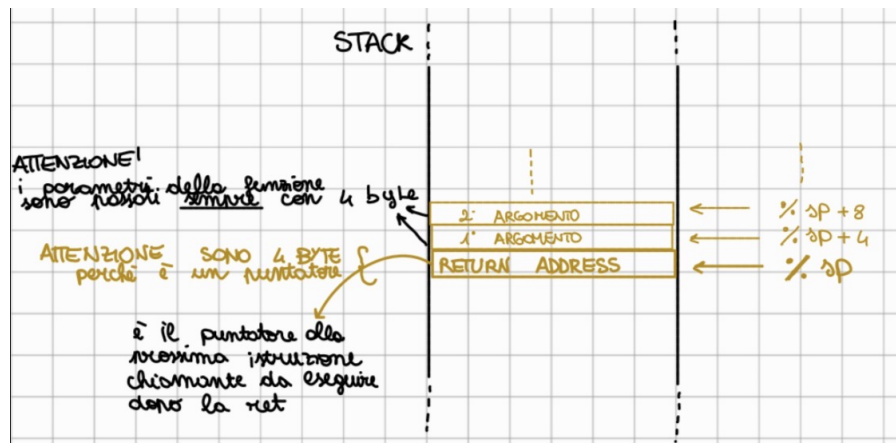


Figura 3.2: StackPointer

Regola: tutti i parametri attuali (argomenti) occupano 4 byte ciascuno nello stack pointer indipendentemente dal loro tipo.

Accesso in memoria con spiazzamento

Per il "passaggio di parametri" in C, abbiamo l'equivalente in assembly in questo modo:

```
d(addr)
```

dove *d* indica lo spiazzamento e *addr* è il puntatore a memoria. Nell'esempio precedente va applicato nel seguente modo:

```
.globl f
f:
    #a -> eax
    movl 4(%esp), %eax
    incl %eax
    ret
```

Quindi avremo che nel registro *eax* sarà salvato il primo parametro passato dalla funzione. In generale avremo che:

```
SP + 4 -> 4(%esp)
```

Operazioni aritmetico/logiche

```
add S, D | D <- D + S
sub S, D | D <- D - S
mul S, D | D <- D * S senza segno
imul S, D | D <- D * S con segno
and S, D | D <- D & S bit a bit
or S, D | D <- D ^ S bit a bit
xor S, D | D <- -D    viene utilizzato per l'inizializzazione delle variabili,
                     preferito a movl poichè modifica i flag
not S, D | D <- !D
inc D    | D <- D + 1
dec D    | D <- D - 1
neg D    | D <- -D
```

Moltiplicazione senza segno

```
mul S | D <- D * S
      |   |   |
      |   |   |__ registro 32bit/immediato
%edx:%eax %eax
```

esempio:

	eax	edx
movl \$2, %eax	0...010	?
mull \$5	0..01010	0.....0

Moltiplicazione con segno

```
imul S | D <- D * S
      |   |   |
      |   |   |__ registro 32bit/immediato
%edx:%eax %eax
```

esempio:

	eax	edx
movl \$-2, %eax	11....10	?
imull \$5	1.....	11111111

Divisione

```
div S | D <- D / S
      |      |
      |      |
      eax,edx  eax:edx

idivl r32/imm32 | eax <- edx:eax / S    // quoziente
                  | edx <- edx:eax % S  // modulo
```

3.3 Gestione del flusso di esecuzione

Salto incondizionato

<pre> goto L; L: </pre>	$\longrightarrow$	<pre> jmp L L: </pre>
---	-------------------	---

c	Assembly
---	----------

Salto condizionato

Le istruzioni di salto condizionato in ASM si sviluppano in due fasi:

1. Esecuzione di istruzione aritmetico/logica che modifica EFLAGS
2. Esecuzione di un salto condizionato al valore di EFLAGS

La proprietà valutata in 2. per il salto è detta **condition code (cc)**

esempio:

Risultato R

Condition Code (cc)	Proprietà di R
e, z	$R == 0$
ne, nz	$R \neq 0$
Con segno:	
g	$R > 0$
l	$R < 0$
ge	$R \geq 0$
le	$R \leq 0$
Senza segno:	
a	$R > 0$
b	$R < 0$
ae	$R \geq 0$
be	≤ 0

Il salto condizionato è implementato da:

```
jcc L
```

esempio:

```
subl %eax, %ecx    # R = ecx - eax
je    L            # eax = ecx -> R==0
```

esempio:

```
subl %eax, %ecx
jge    L            # ecx >= eax -> R>=0
```

Registro EFLAGS

È un registro contenente diversi flag (bit) dedicati a rappresentare lo stato della CPU a seguito dell'esecuzione dell'ultima istruzione:

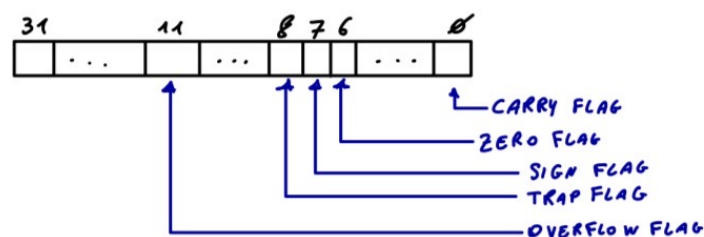


Figura 3.3: EFLAGS

- **Bit 0** → il *carry flag* serve principalmente a indicare un riporto (carry) o un prestito (borrow) nelle operazioni aritmetiche senza segno.
- **Bit 6** → la *zero flag* ha come valore 1 se l'operazione precedente ha avuto come risultato zero, altrimenti varrà 0.
- **Bit 7** → il *sign flag* quando l'operazione ha come risultato negativo vale 1 altrimenti 0
- **Bit 8** → il *trap flag* viene usato dal debugger
- **Bit 11** → l'*overflow flag* indica quando si verifica un overflow nelle operazioni aritmetiche con numeri con segno. Questo succede quando il risultato non può essere rappresentato con il numero di bit disponibile.

7) 8) usato dal debugger 11) per i risultati non rappresentabili nei nostri bit allora il flag varrà 1

3.4 Istruzioni di Condizione

<pre> if(c>a){ istr 1; } istr 2; f.c </pre>	→	<pre> r = c - a; if(r<=0)goto L; istr 1; L: istr 2; f_eq.c </pre>	→	<pre> subl %eax, %ecx jle L #istr 1 L: #istr 2 f.s </pre>
--	---	--	---	---

In questo caso nel codice assembly abbiamo "sporcato" il registro `%ecx`. Per non "sporcare" il registro con SUB usiamo *CMP*, che ha la stessa funzione della sottrazione ma non salva in nessun registro.

```
CMP S, D | D - S
```

Ma altera i valori di EFLAGS esattamente come SUB. Avendo quindi un codice corretto in questo modo:

<pre> if(test){ blocco A; }else{ blocco B; } blocco C; f.c </pre>	→	<pre> if(!test)goto E; blocco A goto F; E: blocco B; F: blocco C; f_eq.c </pre>	→	<pre> CMP S, D jcc E #blocco A jmp F E: #blocco B F: #blocco C f.s </pre>
---	---	---	---	---

Inoltre esiste un'altra funzione simile chiamata *Test* se dobbiamo fare una comparazione a 0, che è equivalente:

```

cmp 0, D | D - 0
test D, D | D & D

```

Cicli While-For

Vedremo ora come fare i cicli in Assembly, cominciando con il *while*. Consideriamo lo schema generale dell'istruzione, se il test effettuato è vero, viene eseguito il blocco e si ritorna al test, altrimenti si prosegue con l'istruzione successiva al while, vediamo un esempio di traduzione in Assembly:

<pre> while(test){ blocco A; } blocco B; f.c </pre>	→	<pre> L: if(!test) goto E; blocco A; goto L; E: blocco B f_eq.c </pre>	→	<pre> L: cmp S, D jcc E blocco A jmp L E: blocco B f.s </pre>
---	---	--	---	---

Quindi non c'è una vera e propria traduzione in Assembly per il while ma occorrerà "smontare" il codice C, in un equivalente in grado di farci giostrare bene i jump, condizionali e non. Vediamo un esempio:

<pre>int sum(int n){ int a = 0; int c = n; while(c>0){ a = a + c; c--; } return a; }</pre> f.c	$\longrightarrow$	<pre>int sum(int n){ int a = 0; int c = n; L: if(c<=0) goto E; a = a + c; c--; goto L; E: return a; }</pre> f_eq.c	$\longrightarrow$	<pre>.global sum sum: xorl %eax, %eax movl 4(%esp), %ecx L: testl \$0, %ecx jle E addl %ecx, %eax decl %ecx jmp L E: ret</pre> f.s
--	-------------------	--	-------------------	---

Consideriamo invece lo schema generale di una istruzione **for**. Si segue dapprima l’inizializzazione e poi si effettua il test, se il test risulta vero allora viene eseguito il blocco, altrimenti si prosegue con l’istruzione successiva al for. In Assembly può essere tradotta in questo modo:

<pre>for(init; test; update){ blocco A } blocco B</pre> f.c	$\longrightarrow$	<pre>init; while(test){ blocco A update; } blocco B</pre> f_eq.c
--	-------------------	---

Quindi passeremo dal for al while corrispondente che siamo già in grado di tradurre. Vediamolo meglio con un esempio:

<pre>unsigned fact(unsigned n){ unsigned a = 1; unsigned c; unsigned d = n; c = 2; L: if(c>d) goto E; a = a * c; c++; goto L; E: return a; }</pre> f.c	$\longrightarrow$	<pre>.globl fact fact: movl \$1, %eax movl \$2, %ecx movl 4(%esp), %edx L: cmpl %edx, %ecx # poichè senza segno # usiamo ja e non jcc ja E imull %ecx, %eax incl %ecx jmp L E: ret</pre> f.s
--	-------------------	---

Quindi per un ciclo for procederemo in questo modo: *for* $\rightarrow$ *while* $\rightarrow$ *if*.

3.5 Puntatori

Un puntatore in C di tipo T^* contiene un indirizzo di memoria dove è conservato un valore di *tipo* T . In Assembly per indirizzare una locazione di memoria il cui indirizzo è contenuto in un registro usiamo l’operatore

<pre> int *a; . . . '(): *a = 0; f.c </pre>	$\longrightarrow$	<pre> #int *a -> eax . . . movl \$0, (%eax) f.s </pre>
--	-------------------	--

Vediamolo in modo grafico:

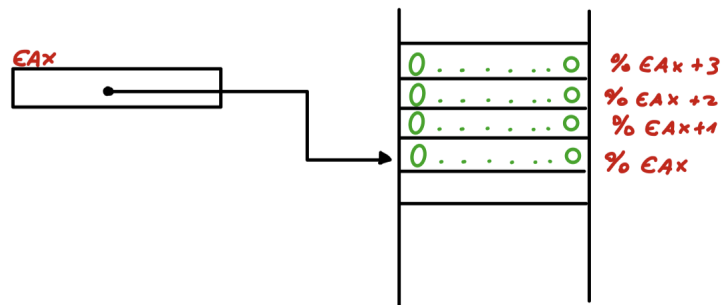


Figura 3.4: Come occupa EAX la memoria in caso di int

Inoltre vale anche: `int c = *a;  $\rightarrow$  movl(%eax), %ecx`

Nel caso dei puntatori la "l" del tipo di dato dipende dalla dimensione del tipo del puntatore (`sizeof(T)`).

Esempio:

<pre> char *p; . . . char a = *p; f.c </pre>	$\longrightarrow$	<pre> #char *p -> edx . . . movb (%edx), %eax f.s </pre>
---	-------------------	--

Quando accediamo al parametro passato per valore ad una funzione, stiamo in effetti usando un puntatore con l'operando '()'. Per capirci meglio:

<pre> void f(int x){ int a = x; } f.c </pre>	$\longrightarrow$	<pre> movl 4(%esp), %eax f.s </pre>
---	-------------------	--------------------------------------

In questi casi noi abbiamo sempre usato dei puntatori verso lo stackpointer:

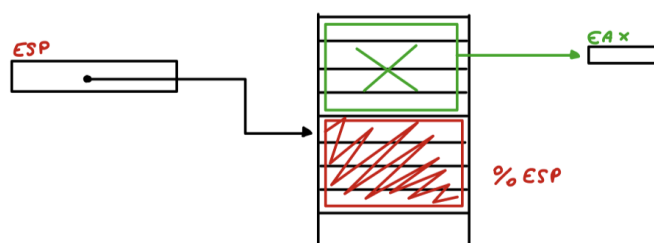


Figura 3.5: Memoria Graficamente

Con lo stesso ragionamento possiamo capire come tradurre gli array in Assembly, molto semplicemente basta accedere alla posizione di memoria corrispondente alla posizione dell'array:

```
int a[7] = 12;                                movl $12, 28(%eax)
f.c                                             f.s
```

Vediamolo meglio però con un esempio:

```
void f(char *x){                               movb 4(%esp), %eax
    char c = x[1];                             movb 1(%eax), %ecx
}
f.c                                             f.s
```

Importante: in una istruzione IA32 è possibile l'accesso a memoria per al più un operando, inoltre l'operatore '()' può essere usato solo sui registri e questi registri devono contenere indirizzi validi di memoria.

Vediamo il tutto con un esempio:

```
void times2(short *p){                         void times2(short *p){           .globl times2
    *p = *p * 2;                               short *a = p;                   times2:
}                                                short cx = *a;                  movb 4(%esp), %eax
                                                *a += cx;                      #essendo short non c'è
                                                                              #bisogno della 'e'
                                                                              movw (%eax), %cx
                                                                              #equivalente fare:
                                                                              #imulw $2, %ecx
                                                                              addw $cx, $cx
                                                                              movw $cx, (%eax)
                                                                              ret
f.c                                             f_eq.c                           f.s
```

Accesso in memoria con base, indice e spiazzamento

$D(x) \rightarrow$ accede alla locazione con indirizzo $x + D$, in realtà la sintassi completa dell'operatore '()' è:

`imm(base, indice, scala)`

dove:

- *imm* $\rightarrow$ rappresenta lo spiazzamento definito come valore costante
- *base* $\rightarrow$ è l'indirizzo di memoria di base a cui deve accedere. **DEVE** essere un registro e **DEVE** essere valido
- *indice* $\rightarrow$ indice dell'elemento a cui si vuole accedere a partire da BASE e **DEVE** essere un registro opzionale
- *scala* $\rightarrow$ dimensione in byte di ogni elemento. **DEVE** essere un valore costante: 1, 2, 4 ed 8 (l'8 solo a 64 bit). Ed è opzionale

L'uso dell'operatore permette di accedere alla memoria con **indirizzo** $\rightarrow$ Base + Indice $\times$ Scala + Imm

Esempio:

```
int a[7] = 12;                                movl $7, %ecx
                                                movl $12, (%eax, %ecx, 4)
                                                # corrisponde a
                                                # movl $12, 28(%eax)
f.c                                             f.s
```

Vediamo un esempio con un codice in C:

```

short get(short* v,
  unsigned n, unsigned i){
  if(i>=n) return -1;
  return v[i];
}
f.c

```

→

```

short get(short* v,
  unsigned n, unsigned i){
  short *c = v;
  unsigned d = i;
  unsigned a = n;
  short ax;
  if(d<a) goto E;
  ax = -1;
  return ax;
E:
  ax = c[d];
  return ax;
}
f_eq.c

```

→

```

.globl get
get:
  movl 4(%esp), %ecx
  movl 12(%esp), %edx
  movl 8(%esp), %eax
  cmpl %eax, %edx
  jnb E
  movlw $-1, %ax
  ret
E:
  movw (%ecx,%edx,2), %ax
  ret
f.s

```

Il tipo del puntatore definisce inoltre il valore da aggiungere all'indirizzo base. Inoltre opereremo su puntatori **sempre** con operatori a 4 byte. In C esiste un'aritmetica dei puntatori **implicita**, ovvero non dobbiamo preoccuparci di dove puntano i puntatori; mentre in Assembly l'aritmetica dei puntatori è **esplicita**.

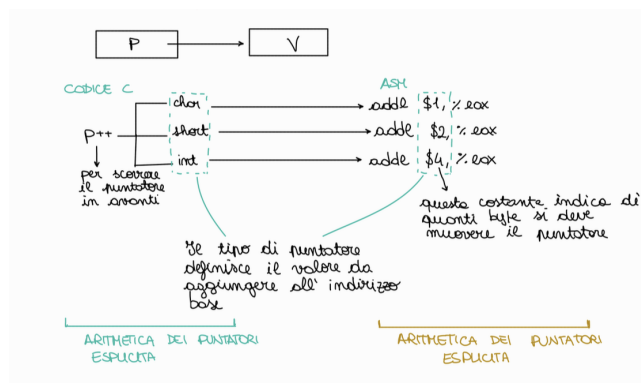


Figura 3.6: Puntatore

3.6 Chiamate a Funzione

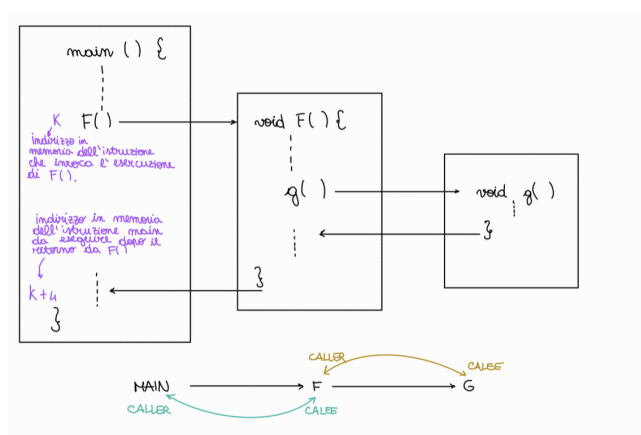


Figura 3.7: Chiamata a funzione

In questo esempio cominciamo definendo queste funzioni. Avremo che il main sarà il nostro **caller**, mentre f sarà il **callee**. Tuttavia, anche tra f e g esiste una relazione: in questo caso f sarà il **caller**, mentre g il **callee**. Quindi una funzione può assumere ruoli diversi a seconda del rapporto che ha con le altre funzioni. La stack è fondamentale per gestire le chiamate a funzione e per fornire spazio di memoria locale. Ogni chiamata a funzione crea uno **stack frame** (o record di attivazione), che contiene variabili locali, parametri e altre informazioni necessarie all'esecuzione della funzione. Gli stack frame sono organizzati come una sorta

di **lista collegata**: il registro ebp punta allo **stack frame** della funzione attualmente in esecuzione, e ogni frame contiene un riferimento a quello della funzione chiamante. Questo permette, ad esempio, a un debugger di ricostruire la sequenza di chiamate dal main fino alla funzione corrente. Tuttavia, il collegamento tra **stack frame** tramite ebp è **opzionale**.

Importante: per convenzione, nel momento in cui si effettua un'istruzione CALL, la base della stack puntata dal registro %esp deve essere sempre a un indirizzo multiplo di 16.

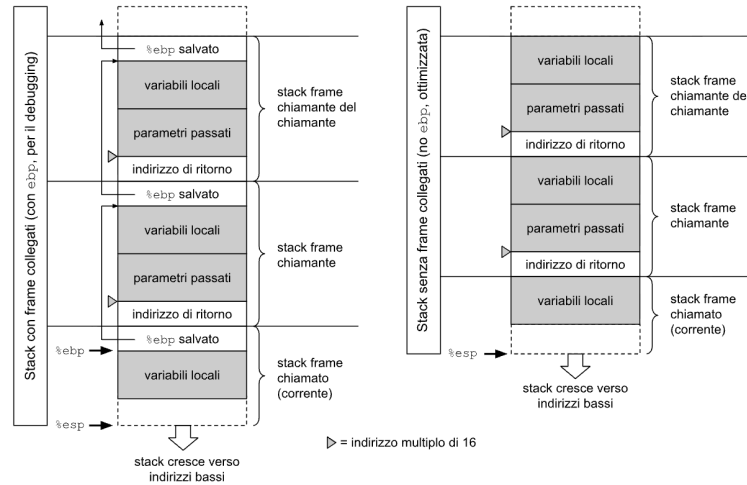


Figura 3.8: Stack Pointer specifico alle funzioni

Vediamo però cosa può contenere il frame/record di attivazione di una funzione:

- Variabili locali
- Copie dei registri $\rightarrow$ ebx, edi, esi, ebp
- Parametri per le funzioni chiamate
- Indirizzo di ritorno delle funzioni chiamate

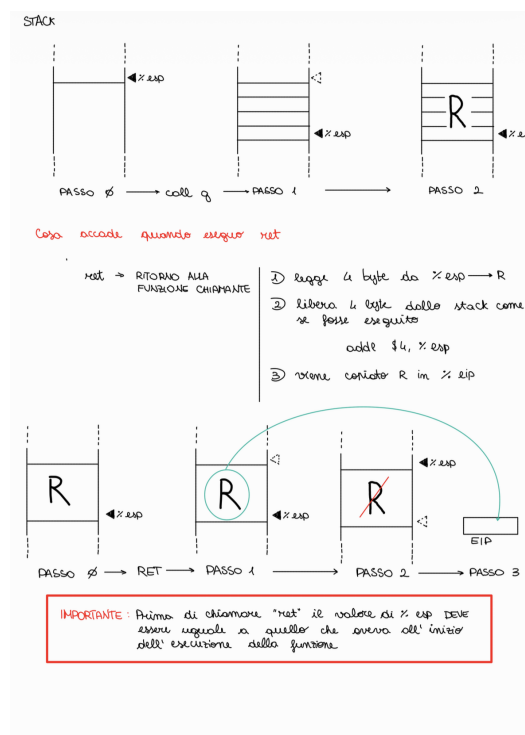


Figura 3.9: Cosa accade quando utilizziamo il ret?

Importante: prima di chiamare ret il valore %esp deve essere uguale a quello che aveva all'inizio dell'esecuzione

<pre>int f(){ int a = 1 + g(); //g è definita nel main //che qui non scriviamo //g () { return 10 } return a; } f.c</pre>	<pre>int f(){ int a; a = g(); a = a + 1; return a; } f_eq.c</pre>	→	<pre>.globl f f: #e -> eax call g #il valore è già in eax #essendo il ret di g incl %eax ret f.s</pre>
--	--	---	--

Vediamo invece cosa accade nello Stack Pointer quando abbiamo un passaggio di parametri

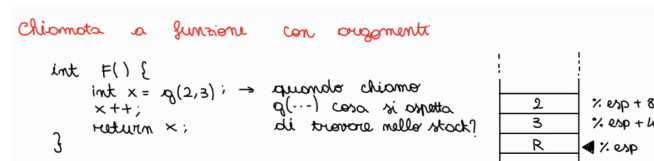


Figura 3.10: Passaggio parametri

Quindi avremo che lo SP:

1. Riserva spazio per i parametri come se fosse: `subl $8, %esp`, dove 8 è uguale al numero di parametri passati per 4 (spazio occupato in memoria)
2. Copia i valori attuali dei parametri nello spazio riservato in ordine inverso
3. Chiamata di funzione: `call g`

<pre>int f(){ int x = g(2,3); //g è definita nel main //che qui non scriviamo x++; return x; } f.c</pre>	→	<pre>.globl f f: #8 perchè 4 * 2 parametri subl \$8, %esp #prologo #scrivo i parametri attuali #nello spazio che ho riservato movl \$3, 4(%esp) movl \$2, 8(%esp) call g #dato che se g restituisce qualcosa #lo restituisce in eax avremo #eax già modificato incl %eax #riportiamo %esp al valore "originale" addl \$8, %esp #epilogo ret f.s</pre>
---	---	--

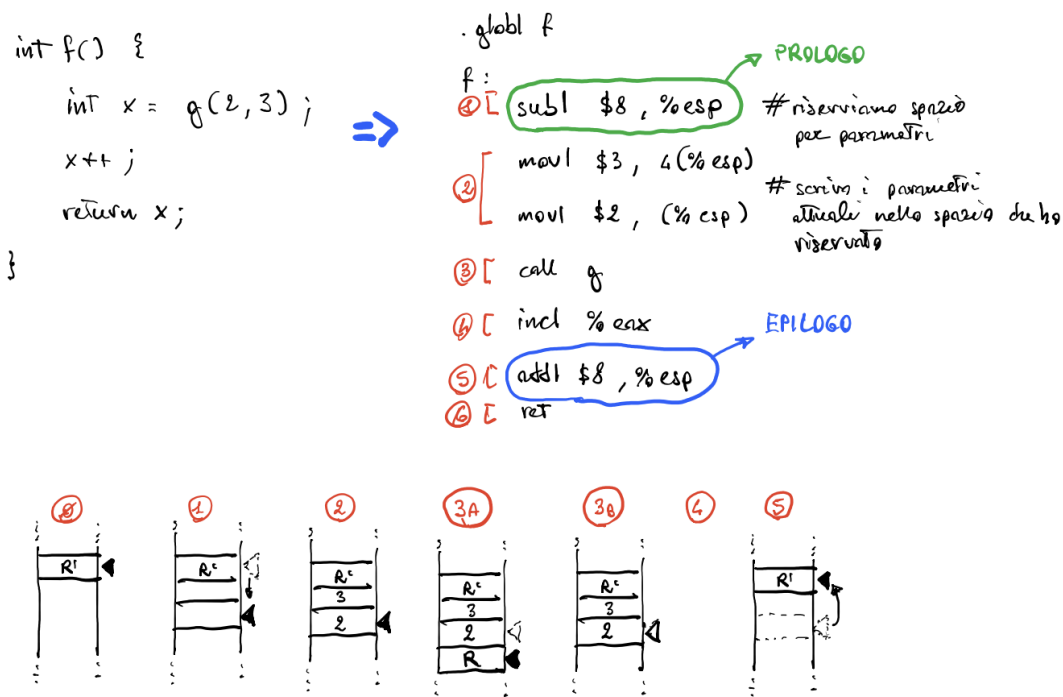


Figura 3.11: Struttura del codice

La lista di stack frame viene gestita mediante un codice di **prologo** all'inizio di una funzione e un codice di **epilogo** alla fine:

- *prologo* → salva nello stack il contenuto del registro **ebp**, che punta allo stack frame del chiamante, tramite l'istruzione `pushl %ebp`. Successivamente il registro **ebp** viene aggiornato per puntare alla posizione corrente dello stack, contenuta nel registro **esp**, mediante l'istruzione `movl %esp, %ebp`. In questo modo **ebp** viene a puntare allo stack frame della funzione corrente.
- *epilogo* → ripristina il valore del registro **ebp** presente prima dell'attivazione della funzione corrente eseguendo l'istruzione `popl %ebp`. In questo modo **ebp** tornerà a puntare allo stack frame della funzione chiamante.

In gcc, è possibile omettere il collegamento fra stack frame compilando con l'opzione `-fomit-frame-pointer`. In questo modo, non verranno generati prologo ed epilogo: la funzione sarà più veloce e compatta, ma il debugging potrebbe essere più difficoltoso.

L'esecuzione di una funzione può sovrascrivere i registri utilizzati dal chiamante. Se il chiamante vuole garantire che il contenuto di un registro non venga modificato durante l'invocazione di una funzione, è necessario salvarne il valore da qualche parte, generalmente nello stack. Esistono due possibili strategie:

1. **caller-save**: il registro viene salvato nello stack dal chiamante prima dell'invocazione della funzione e ripristinato subito dopo il ritorno. La funzione chiamante (caller) deve salvare il valore di questi registri prima di chiamare un'altra funzione. La funzione chiamata invece può modificare liberamente il valore di questi registri.
2. **callee-save**: il registro viene salvato nello stack dal chiamato prima di eseguire il corpo della funzione e ripristinato prima di ritornare al chiamante. La funzione chiamata (callee) deve preservare il valore che questi registri riservano nel momento in cui viene eseguita la call, e deve ripristinarli prima di eseguire il `ret` (il salvataggio avviene nel prologo e il ripristino nell'epilogo).

Per convenzione, alcuni registri vengono salvati dal chiamante, e altri dal chiamato:

- **Registri caller-save** → A, C, D.
- **Registri callee-save** → B, DI, SI, SP, BP.

Ma come si salva il valore di un registro?

1. Salviamo il valore di `%reg` (registro qualsiasi) nella stack
2. Eseguiamo le operazioni che "cambiano" il contenuto di `%reg`

3. Recuperiamo dalla stack il valore di %reg

Ci sono diversi modi per farlo, vediamo il *modo esplicito*:

- 1) `subl $4, %esp`
`movl %reg, (%esp)`
- 2) `#%reg` viene sporcato
- 3) `movl (%esp), %reg`
`addl $4, (%esp)`

Metodo *Push + Pop*

- 1) `push %reg`
- 2) `# . . .`
- 3) `popl %reg`

In questi due metodi abbiamo che l'1) e il 3) sono equivalenti tra loro quindi possiamo usare entrambi a nostro piacimento.

<pre>int f(){ return g() + h(); }</pre>	→	<pre>int f(){ int a; a = g(); int c; c = a; a = h(); a = a + c; return a; }</pre>	→	<pre>.globl f f: #a -> EAX #c -> ECX call g movl %eax, %ecx pushl %ecx call h popl %ecx addl %ecx, %eax ret f.s</pre>
<code>f.c</code>		<code>f_eq.c</code>		

Avremmo potuto usare anche il metodo esplicito ed avremmo avuto lo stesso identico risultato.

Vediamo però che registri usare a seconda delle situazioni:

- Se implemento una funzione che **NON** chiama altre funzioni:
 - uso liberamente i caller saved (A, C, D).
 - se non sono sufficienti allora uso i callee saved (B, DI, SI, BP) ma preservando il valore (usando prologo + epigolo).
- Se implemento una funzione che chiama altre funzioni:
 - Uso i callee saved (B, DI, SI, BP) preservandone il valore
 - Uso liberamente i caller saved (A, C, D) per valori temporanei **SOLO** in blocchi di istruzioni che **NON** chiamano altre funzioni,
 - Altrimenti uso i caller saved (A, C, D) ma ne preservo il valore (push + pop) prima di **QUALSIASI** chiamata a funzione.

```

int g(int x);

int f(){
    int y = 1, z = 2;
    return g(x+1) + y + z;
}

→                                     →

int f(){
    int c;
    int di; // y
    int si; // z
    c = x;
    di = 1;
    si = 2;
    c++;
    int a = g(c);
    a = a+di;
    a = a + si;
    return a;
}

                                     .globl f
f:
    #Prologo
    pushl %edi # -> y
    pushl %esi # -> z
    subl $4, (%esp)

    movl 16(%esp), %ecx
    movl $1, %edi
    movl $2, %esi
    incl %ecx
    movl %ecx, (%esp)
    call g
    addl %edi, %eax
    adll %esi, %eax
    #Epilogo
    addl $4, (%esp)
    popl %edi
    popl %esi
    ret

f.c                                     f_eq.c                                     f.s

```

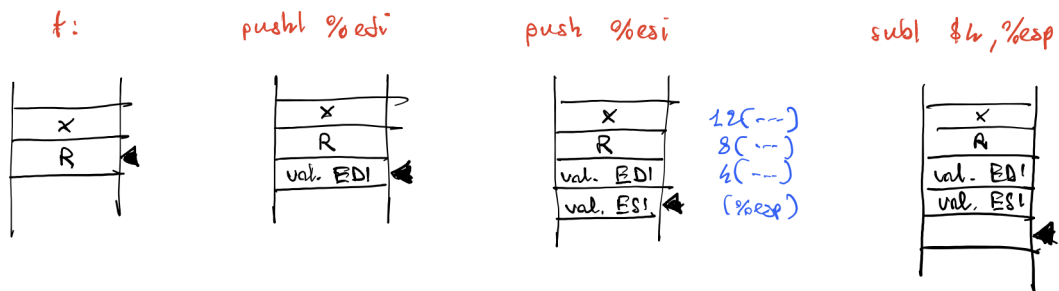


Figura 3.12: Gráficamente

Esercizio riassuntivo su quello fatto fino ad ora:

```
int is_negative(int x);

int f(int *v, unsigned n){
    unsigned i, cnt = 0;
    for(i = 0; i < n; i++){
        cnt += is_negative(v[i]);
    }
    return cnt;
}

f.c
```

<pre>int f(int *v, unsigned n){ int *di = v; unsigned si = n; unsigned b = 0; unsigned bp = 0; L: if(b >= si) goto E; int c = di[b]; unsigned a = is_negative(c); bp = bp + a; b++; goto L; E: a = bp; return a; } f_eq.c</pre>	→	<pre>.globl f f: pushl %edi pushl %esi pushl %ebx pushl %ebp subl \$4, %esp movl 24(%esp), %edi movl 28(%esp), %esi xorl %ebx, %ebx xorl %ebp, %ebp L: cmpl %esi, %ebx jae E movl (%edi,%ebx,4), %ecx #4 * ebx (=i) quindi muove di 4 * i %edi #per prendere tutti i valori del vettore movl %ecx, (%esp) call is_negative addl %eax, %ebp incl %ebx jmp L E: movl %ebp, %eax ret</pre>
---	---	--

3.7 Conversione del tipo (cast)

Partiamo dal fatto che in Assembly il cambio di tipo, così come lo intendiamo nei linguaggi ad alto livello, non esiste. In questo contesto dobbiamo invece distinguere due casi principali:

1. **Operazioni di troncamento (downcast)** → ad esempio: `int c = 0x000CAFE` (32 bit) → `char a = c` (8 bit). In questo caso si mantengono solo i bit meno significativi, con conseguente perdita di informazione. (Esempio implementabile con istruzioni come `movl`).
2. **Operazioni di promozione (upcast)** → in questo caso il valore viene esteso a un tipo più grande. È importante distinguere tra estensione con segno e senza segno, a seconda che il dato originale sia signed oppure unsigned.

con		<code>char c='s';</code>	8 bit
segno		<code>int a=c;</code>	32 bit
senza		<code>unsigned char c='s';</code>	8 bit
segno		<code>unsigned int a=c;</code>	32 bit

Con questo tipo di operazione non si ha perdita di informazione, a differenza del downcast. Tuttavia, è necessario garantire esplicitamente che il segno venga preservato.

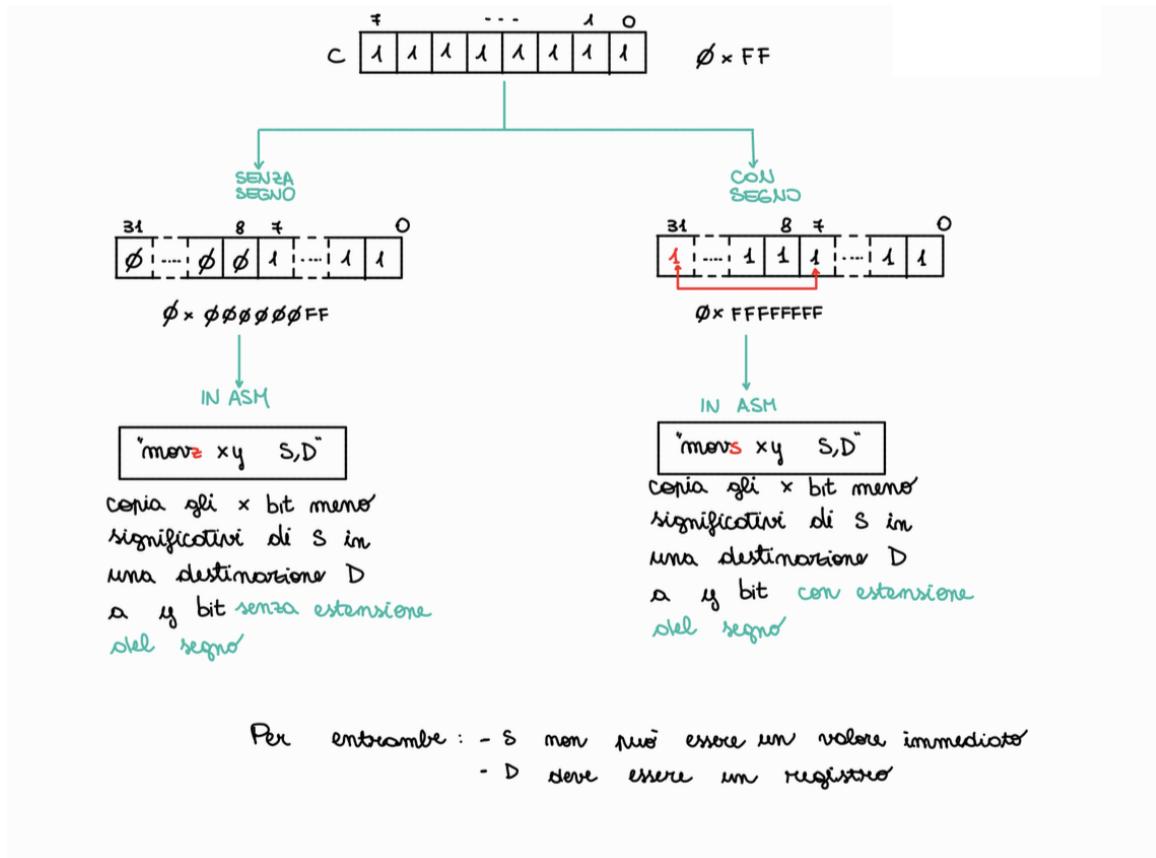


Figura 3.13: Estensione con segno da char a int

Nel caso di una promozione da `char` a `int`, per preservare il segno è necessario propagare il bit più significativo del `char` (bit di segno) su tutti i bit aggiuntivi dell'`int`.

Per questo motivo si distinguono due tipi di istruzioni:

- estensione con segno (*sign extension*);
- estensione senza segno (*zero extension*).

movs xy S, D -> copia gli x bit meno significativi di S in una destinazione D a y bit
con estenzione del segno
movz xy S, D -> copia gli x bit meno significativi di S in una destinazione D a y bit
senza estenzione del segno

Per entrambe vale che: S non può essere un valore immediato, e D deve essere un registro. Per le operazioni con segno utilizzeremo i seguenti comandi:

	char c	short c	int c
char r	movb %cl, %al		
short a	movsbw %cl, %ax	movw %cx, %ax	
int a	movsbl %cl, %eax	movswl %cx, %eax	movl %ecx, %eax

Per le operazioni senza segno:

	char c	short c	int c
char r	movb %cl, %al		
short a	movzbw %cl, %ax	movw %cx, %ax	
int a	movzbl %cl, %eax	movzwl %cx, %eax	movl %ecx, %eax

esempio:

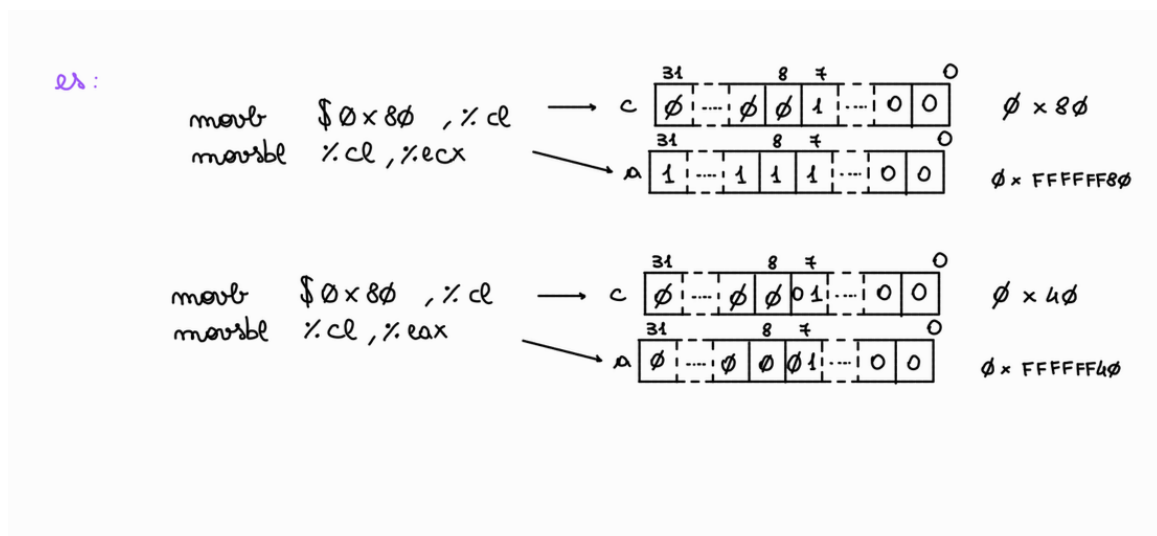


Figura 3.14: Esempio

Esempio:

```

int f(char *p, char *q){
    int x = *p + *q;
    return x
}

```

f.c

```

.global f
f:
    movl 4(%esp), %eax # A <-> p
    movl 8(%esp), %ecx # C <-> q
    movsbl (%eax), %eax
    movsbl (%ecx), %ecx
    addl %ecx, %eax
    ret
f.s

```

Altro esempio:

<pre> int is_vowel(int c); // scritto nel mai e // restituisce 0/1 int num_vowels(char *s){ int cnt = 0; while(*s) cnt += is_vowel(*s++); return cnt; } </pre>	→	<pre> int is_vowel(int c); int num_vowels(char *s){ char *b = s; int di = 0 // di <-> cnt L: if(*b == 0) goto E; int a = *b; //ATTENZIONE: promozione a = is_vowel(a); di = di + a; b++; goto L; E: a = di; return a; } </pre>	→	<pre> .globl num_vowels num_vowels: #Prologo pushl %ebx pushl %edi subl \$4(%esp) movl 16(%esp), %ebx xorl %edi, %edi L: cmpb \$0, (%ebx) je E movsbl %ebx, %eax movl %eax, (%esp) call is_vowel addl %eax, %edi incl %ebx jmp L E: movl %edi, %eax #Epilogo addl \$4(%esp) popl %edi popl %ebx ret </pre>
f.c		f_eq.c		f.s

Variabili Locali

Quando possibile, il compilatore utilizza i registri della CPU per memorizzare i valori delle variabili locali, poiché questa soluzione è la più semplice e garantisce le migliori prestazioni in termini di velocità di accesso. Tuttavia, non sempre ciò è possibile: in alcuni casi le variabili devono essere allocate nello *stack frame* della funzione, cioè in memoria. Questo accade nei seguenti casi:

1. **Registri insufficienti:** se il numero di variabili locali utilizzate dalla funzione supera quello dei registri disponibili, il compilatore è costretto a memorizzarne alcune nello stack.
2. **Uso dell'operatore &:** quando si applica l'operatore di indirizzo a una variabile locale, questa deve necessariamente risiedere in memoria, poiché deve avere un indirizzo ben definito.
3. **Variabili complesse:** se la variabile è un array o una struttura (**struct**), non può essere contenuta interamente in un registro e quindi viene allocata nello stack.

In sintesi, l'uso dei registri rappresenta un'ottimizzazione importante, ma è vincolato sia da limitazioni hardware (numero di registri) sia da requisiti semantici del programma (necessità di un indirizzo o dimensioni dei dati).

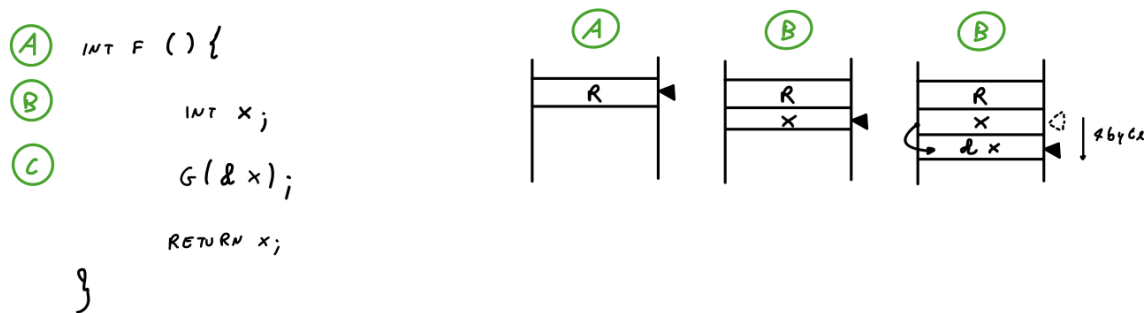


Figura 3.15: Caption

Questo in assembly:

```
.globl f
f:
    #Prologo
    subl $8, %esp
    movl %esp, (%eax)    #lo salviamo in un registro
    addl $4, %eax        #poichè 'addl $4, (%esp)' non è corretto
    #quindi per modificare il valore dovremmo prima portarlo in un altro registro
    movl %eax, (%esp)
    call g
    movl 4(%esp), %eax
    #Epilogo
    addl $8, %esp
    ret
```

Nel dettaglio, la seguente sequenza di istruzioni:

```
movl %esp, (%eax)
addl $4, %eax
```

può essere riscritta in modo più compatto utilizzando l'istruzione `leal`:

```
leal 4(%esp), %eax
```

L'istruzione **LEA** (Load Effective Address) consente di calcolare un indirizzo di memoria senza accedere effettivamente al contenuto di quella locazione. Si limita quindi a eseguire un calcolo aritmetico e a salvare il risultato in un registro. La sua forma generale è:

LEA $O(B, I, S)$, D

dove l'indirizzo calcolato è:

$$B + I \cdot S + O$$

In altre parole, il registro di destinazione viene aggiornato come segue:

- inizialmente assume il valore base B
- viene aggiunto l'offset O
- viene aggiunto il contributo dell'indice $I \cdot S$

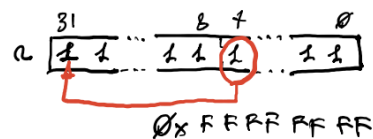
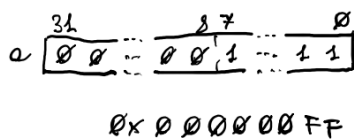
Per chiarire la differenza rispetto a mov, consideriamo il seguente confronto:

leal 4(%esp), %eax	movl 4(%esp), %eax
calcola ESP + 4 e scrive il risultato in EAX	→ calcola ESP + 4, accede alla memoria in quell'indirizzo e copia il valore in EAX

Quindi, mentre LEA esegue solo un calcolo di indirizzi, MOV comporta anche un accesso alla memoria.

Importante: Se usiamo LEA per calcoli aritmetici lo status register (EFLAGS) NON viene alterato. Quindi non è da usare per le jump condizionate.

void get_div_rem(int *d, int *r, int x, int y);	.globl f
	f:
	#Prologo
	subl \$24, %esp
	movl 28(%esp), %eax #x
	movl 32(%esp), %ecx #y
void get_div_rem(int *d, int *r, int x, int y);	leal 16(%esp), %edx #&d
	movl %edx, 4(%esp)
int f(int x, int y){	#stessa cosa per il secondo parametro
int d, r;	leal 20(%esp), %edx #&r
get_div_rem(&d, &r, x, y);	movl %edx, 4(%esp)
return d;	
}	
	movl %eax, 8(%esp)
	movl %ecx, 12(%esp)
	call get_div_rem
	movl 16(%esp), %eax
	#Epilogo
	addl \$24, %esp
	ret
f.c	f.s



In ASM eseguiamo l'insert con due istruzioni diverse:

mov **s** **x** **y** S, D

copia gli **x** bit meno significativi
di S in una destinazione D a
y bit con estensione del segno

mov **z** **x** **y** S, D

copia gli **x** bit meno significativi
di S in una destinazione D a
y bit senza estensione del segno

Per entrambe: S non può essere un valore immediato
D deve essere un registro

Figura 3.16: Graficamente

Espressioni Booleane

L'istruzione `SETcc` (dove `cc` sta per *condition code*) consente di assegnare il valore 0 oppure 1 a un registro a 8 bit oppure a un byte in memoria, in base al risultato di una condizione valutata sui flag contenuti nel registro `EFLAGS`. La forma generale dell'istruzione è:

`SETcc D`

Il suo comportamento può essere descritto come:

$$D \leftarrow \text{condizione}$$

In particolare:

- se la condizione associata al suffisso `cc` è verificata, viene scritto il valore 1 in `D1`;
- altrimenti, viene scritto il valore 0.

Questa istruzione è spesso utilizzata per trasformare il risultato di un confronto in un valore booleano (0 o 1), utile ad esempio nelle espressioni condizionali.

<pre>int f(int x,int y){ return x<y; }</pre>	$\longrightarrow$	<pre>int f(int x,int y){ int a = 1; if(x<y) goto E; a = 0; E: return a; }</pre>	$\longrightarrow$	<pre>.globl f f: movl 4(%esp), %eax #x movl 8(%esp), %edx #y cmpl %edx, %ecx setl %al #restituisce un int movzbl %al, %eax ret</pre>
---	-------------------	--	-------------------	--

Altro esempio:

<pre>int between(short x,short y, short z){ return x<=y && y<=z; }</pre>	$\longrightarrow$	<pre>.globl between between: #Prologo pushl %ebx movw 8(%esp), %bx #x movw 12(%esp), %cx #y movw 16(%esp), %dx #z cmpw %cx, %bx setle %al cmpw %dx, %cx stele %ah andb %ah, %al movzbl %al, %eax #Epilogo popl %ebx ret f.s</pre>
--	-------------------	---

Regola del cortocircuito: differenza tra `&` e `&&` in C

In C esistono due operatori che possono sembrare simili ma hanno comportamenti diversi:

- `&&` (AND logico): applica la **valutazione con cortocircuito**. La seconda espressione viene valutata **solo se** la prima è vera (diversa da zero). Se la prima è falsa, il risultato è già determinato e la seconda non viene eseguita.
- `&` (AND bit a bit): **non utilizza il cortocircuito**. Entrambe le espressioni vengono sempre valutate, indipendentemente dal valore della prima.

In sintesi, il cortocircuito è una proprietà dell'operatore logico `&&`, mentre l'operatore `&` esegue sempre entrambe le valutazioni.

3.8 Allineamento nello Stack

Consideriamo il seguente esempio:

<code>void g(char *c);</code>		<code>f:</code>
		<code>subl \$4, %esp # spazio per a (allineato a 4 byte)</code>
<code>void f(){</code>		<code>subl \$4, %esp # spazio per il parametro di g</code>
<code>...</code>	$\longrightarrow$	<code>...</code>
<code>char a; // 1 byte</code>		<code>leal ...</code>
<code>g(&a); // parametro: 4 byte</code>		<code>movl ..., (%esp)</code>
<code>...</code>		<code>call g</code>
<code>}</code>		<code>...</code>

Importante: In ogni momento dell'esecuzione, lo **stack pointer** deve essere allineato a un multiplo di 4 byte. In pratica, i compilatori moderni allineano lo stack a **16 byte**.

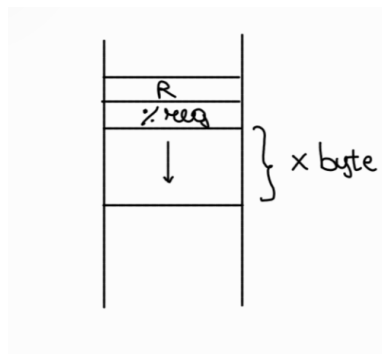


Figura 3.17: SP

Esempio con più variabili:

	<code># Prologo</code>
	<code>subl \$16, %esp # spazio totale</code>
<code>char a, b, c;</code>	
<code>g(&a, &b, &c);</code>	$\longrightarrow$ <code># Spiegazione:</code>
	<code># - a, b, c occupano 1 byte ciascuno</code>
	<code># - per allineamento diventano 4 byte ciascuno</code>
	<code># - parametri di g: 3 * 4 byte = 12 byte</code>
	<code># - totale: 3*4 + 12 = 24 → arrotondato a 16 (allineamento)</code>

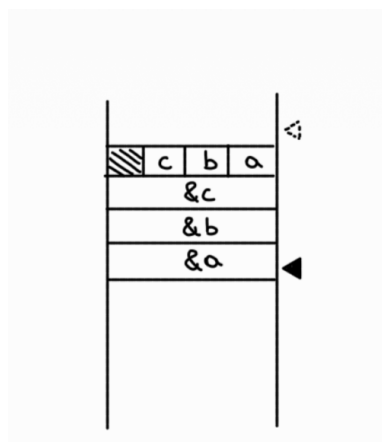


Figura 3.18: Ecco cosa succede nello SP

Regola fondamentale: Un dato di dimensione x viene allocato a indirizzi multipli di x .

Altro esempio:

<pre>char a[5]; short b; g(&a, &b); f.c</pre>	→	<pre># Prologo subl \$16, %esp # Spiegazione: # - a occupa 5 byte → allineato a 8 byte # - b occupa 2 byte → allineato a 4 byte # - parametri: 2 * 4 = 8 byte # - totale: 8 + 4 + 8 = 20 → arrotondato a 16/32 f.s</pre>
--	---	--

3.9 Allineamento delle Struct

Per tradurre una **struct** in Assembly, bisogna rispondere a:

1. Dove inizia ogni campo in memoria?
2. Quanto spazio occupa complessivamente la struct?

Regole di allineamento:

1. Un dato di dimensione x deve essere allineato a indirizzi multipli di x
2. L'allineamento della struct è determinato dal campo più grande
3. La dimensione totale della struct è un multiplo della dimensione del campo più grande

Esempio (non ottimizzato)

```
struct s{
    char x;    // +0 |x - - -|
    int y;     // +4 |x x x x|
    short z;   // +8 |x x - -|
                // sizeof(s) = 12 (5 byte sprecati)
};
```

Esempio (ottimizzato):

```
struct s{
    char x;    // +0 |x -
    short z;   // +2 |    x x|
    int y;     // +4 |x x x x|
                // sizeof(s) = 8 (1 byte sprecato)
};
```

Nota: Riordinare i campi riduce lo spazio sprecato e migliora l'efficienza.

Esempio completo con Assembly:

<pre>struct s{ char x; // +0 int y; // +4 }; void f(struct s *p, char a, int b){ p->x = a; p->y = b; }</pre>	→	<pre>.globl f f: movl 4(%esp), %eax # p movb 8(%esp), %cl # a movl 12(%esp), %edx # b movb %cl, (%eax) # p->x movl %edx, 4(%eax) # p->y ret</pre>
--	---	--

3.10 Istruzioni SHIFT

Le istruzioni di shift (spostamento) in Assembly consentono di spostare i bit di un operando a sinistra o a destra. Esistono due tipi principali di shift:

- **Shift con segno**

SAL S,D | D <- D << S I nuovi bit meno significativi sono posti uguali a 0
 SAR S,D | D <- D >> S I nuovi bit più significativi sono posti uguali al
 valore del bit più significativo

- **Shift senza segno**

SHL S,D | D <- D << S I nuovi bit meno significativi sono posti pari a 0
 SHR S,D | D <- D >> S I nuovi bit meno significativi sono posti pari a 0

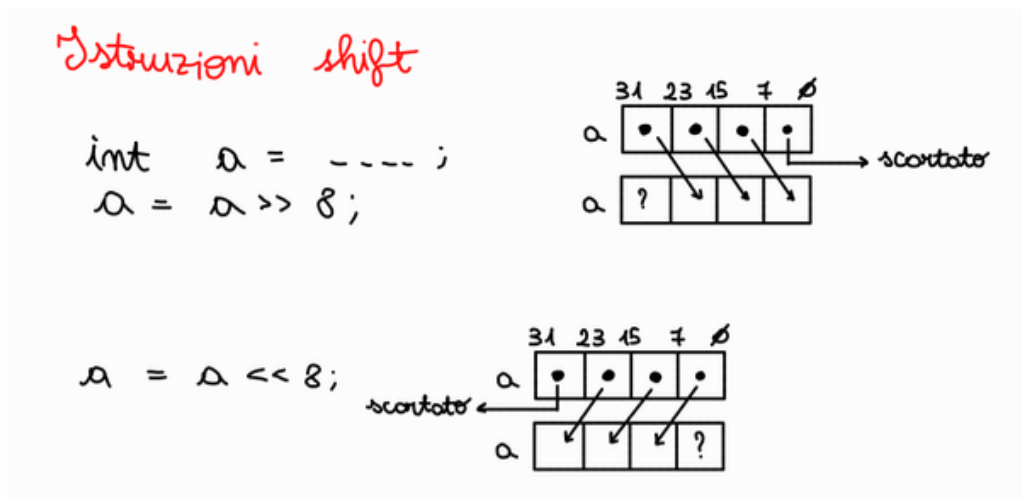


Figura 3.19: Shift dei bit

Esempio:

```
int c = 0xABADCAFE;
unsigned d = 0xABADCAFE;
c = c << 8;      // assembly: sall $8,%ecx      %ecx -> 0xADCAFE00
d = d << 8;      // assembly: shll $8,%ecx      %ecx -> 0xADCAFE00
c = c >> 8;      // assembly: sarl $8,%ecx      %ecx -> 0xFFADCAFE
d = d >> 8;      // assembly: shrll $8,%ecx     %ecx -> 0x00ADCAFE
```

Esempio:

```
int a = 8;      000...001000 -> 8
a = a >> 2;     000...000010 -> 2 = 8/(2^2) esponente di 2 è il numero di bit shift
```

$a \gg x \rightarrow a/2^x$

```
int a = 8;      000...001000 -> 8
a = a << 2;     000...100000 -> 32 = 8*(2^2) esponente di 2 è il numero di bit shift
```

$a \ll x \rightarrow a \cdot 2^x$

Vediamo ora attraverso un esercizio:

<pre> unsigned count1(unsigned n){ unsigned a=0; while(n>0){ if(n & 1)a++; n = n >> 1; } return a; } </pre> f.c	<pre> unsigned count1(unsigned n){ unsigned eax=0; unsigned ecx=n; L: if(c <= 0) goto E; if ((n & 1)==0) goto A; eax++; A: ecx = ecx >> 1; goto L; E: return eax } </pre> f_eq.c	<pre> .globl count1 count1: xorl %eax,%eax movl 4(%esp),%ecx L: cmpl \$0,%ecx jbe E testl \$1,%ecx jz A incl %eax A: shrl \$1,%ecx jmp L E: ret </pre> f.s
---	--	---

3.11 Istruzione IDIV

L'istruzione IDIV esegue la divisione intera con segno tra il valore contenuto nel registro EAX (dividendo) e un operando specificato (divisore). Il risultato della divisione viene memorizzato in due registri:

- Il quoziente (risultato della divisione) viene memorizzato in EAX.
- Il resto (modulo) viene memorizzato in EDX.

```

idiv s | A <- D : A / S quoziente
       | D <- D : A % S resto

```

Esempio:

<pre> int f(int x, int y){ return x/y; } </pre> f.c	<pre> int f(int x, int y){ int a = x; int c = y; int d = a; d=d >> 31; a = a / c; return a; } </pre> f_eq.c	<pre> .globl f f: movl 4(%esp),%eax movl 8(%esp),%ecx movl %eax,%edx sarl \$31,%edx idivl %ecx ret </pre> f.s	questa coppia di istruzioni prepara per i idiv
--	--	--	---

Quando usiamo idiv, la divisione non avviene solo su EAX, ma su EDX:EAX (un numero a 64 bit). Quindi dobbiamo prima preparare EDX correttamente: deve contenere l'estensione del segno di EAX. Per fare ciò utilizziamo lo shift aritmetico a destra (SAR) di 31 bit, che riempie EDX con il bit più significativo di EAX (0 se EAX è positivo, 1 se è negativo). In questo modo, quando eseguiamo IDIV, otteniamo il risultato corretto anche per numeri negativi. Quindi:

```

movl %eax,%edx
sarl $31,%edx

```

serve proprio a questo: copia il bit di segno di EAX in tutto EDX

- EDX = 0 se il numero è positivo
- EDX = -1 se è negativo

3.12 Istruzione CMOV

L'istruzione CMOVcc (Conditional Move) consente di eseguire un'operazione di assegnamento condizionale, spostando il valore da una sorgente a una destinazione solo se una specifica condizione è soddisfatta. La forma generale dell'istruzione è:

```
CMOVcc S,D | D <- S se cc è vero
            | D <- D se cc è falso
```

S non può essere un valore immediato
D deve essere un registro a 16/32 bit

Esempio

	<pre>int max(int x, int y){ int eax = x; int ecx = y; if(ecx > eax) goto E; eax=ecx; return eax; }</pre>	
int max(int x, int y){ return (x > y) ? x : y; }	→ E:	→
f.c	f_eq.c	f.s

3.13 Ottimizzazione dei programmi

L'ottimizzazione di un programma consiste nel modificarlo in modo da minimizzare l'uso delle risorse disponibili, cercando allo stesso tempo di massimizzare le prestazioni, senza compromettere la correttezza.

Problemi principali:

1. Come valutare l'impatto di una modifica sul programma?
In altre parole: come misurare le prestazioni e l'uso delle risorse?
2. Come individuare le parti del programma da ottimizzare?
Come identificare i *punti caldi* (hot spots) che meritano attenzione?

Misure prestazionali:

- **Latenza:** tempo necessario per eseguire il programma
- **Throughput:** numero di operazioni completate per unità di tempo
- **Spazio:** memoria utilizzata
- **Energia:** consumo energetico

Analisi prestazionale del programma

Ad esempio, quanto tempo richiede l'esecuzione dell'istruzione:

```
movl $10, (%eax) ?
```

Possiamo scomporre l'operazione in:

1. Lettura del registro **eax** → CPU + registri (operazione veloce)
2. Scrittura del valore 10 in memoria → RAM (operazione più costosa)

Il miglioramento prestazionale si misura tramite lo **speedup**:

$$\text{speedup} = \frac{T}{T'}$$

dove T è il tempo di esecuzione originale e T' quello ottimizzato.

Legge di Amdahl

La legge di Amdahl permette di calcolare lo speedup complessivo di un programma quando si ottimizza solo una sua parte.

Sia:

$$T = \alpha T + (1 - \alpha)T$$

dove:

- α è la frazione di tempo spesa nella parte ottimizzabile
- $(1 - \alpha)$ è la parte non ottimizzabile

Supponiamo di ottenere uno speedup k_A sulla parte ottimizzabile. Allora:

$$T' = \frac{\alpha T}{k_A} + (1 - \alpha)T$$

Lo speedup complessivo diventa:

$$S = \frac{T}{T'} = \frac{1}{\frac{\alpha}{k_A} + (1 - \alpha)}$$

Procedimento:

$$S = \frac{T}{T'} \quad T' = T'_A + T'_B = \frac{T_A}{k_A} + T_B = \frac{\alpha T}{k_A} + (1 - \alpha)T = (\frac{\alpha}{k_A} + 1 - \alpha)T$$

$$S = \frac{T}{T'} = \frac{T}{(\frac{\alpha}{k_A} + 1 - \alpha)T} = \frac{1}{\frac{\alpha}{k_A} + 1 - \alpha}$$

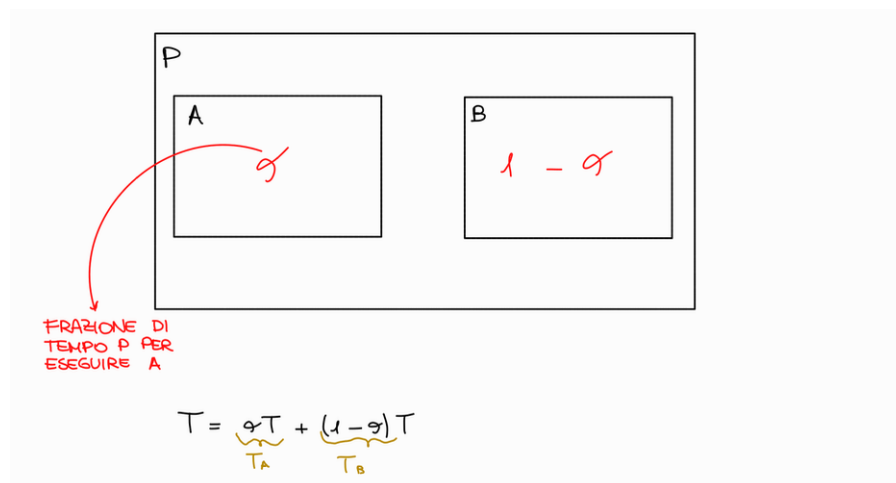


Figura 3.20: Legge di Amdahl

Esempi

- $\alpha = 0.5, k_A = 2 \Rightarrow S \approx 1.33$
- $\alpha = 0.9, k_A = 6 \Rightarrow S \approx 4$

Limite teorico Qual è lo speedup massimo ottenibile?

$$\lim_{k_A \rightarrow \infty} \frac{1}{\frac{\alpha}{k_A} + (1 - \alpha)} = \frac{1}{1 - \alpha}$$

Questo mostra che lo speedup è limitato dalla parte non ottimizzabile del programma.

3.14 Gestione dinamica della memoria in Assembly con malloc

In assembly, la gestione della memoria dinamica non è automatica come in C: il programmatore deve gestire esplicitamente la chiamata alla funzione `malloc`, il controllo degli errori e l'utilizzo del puntatore restituito.

Ruolo di malloc

La funzione `malloc` è una funzione di libreria che:

- Alloca un blocco di memoria nello heap
- Restituisce l'indirizzo del blocco allocato
- Restituisce NULL (0) in caso di errore

In assembly, non esiste alcun meccanismo automatico: `malloc` viene trattata come una normale funzione esterna.

Gestione interna in assembly

Quando si usa `malloc` in assembly, bisogna gestire manualmente:

1. **Preparazione del parametro:** → la dimensione da allocare (in byte) deve essere calcolata dal programmatore. Ad esempio, per una struct:

```
sizeof(node_t) = 2 (short) + 4 (pointer) + 2 (padding)
               = 8 byte
```

Il padding sarebbe memoria vuota per garantire l'allineamento corretto dei campi.

2. **Preparazione dello Stack:** → viene riservato lo spazio nello stack per l'allocazione della grandezza (8 = 4 byte): (riprendendo l'esempio di sopra)

```
subl $4, %esp
```

3. **Chiamata della funzione**

```
call malloc
```

4. **Recupero del risultato** → il puntatore restituito è contenuto nel registro **EAX**:

```
eax = puntatore al blocco allocato
```

5. **Controllo dell'errore** → è necessario verificare se il puntatore è nullo:

```
cmpl $0, %eax
je errore
```

6. **Uso della memoria allocata** → il puntatore in **EAX** viene usato per accedere e inizializzare la struttura:

```
movl %val, (%eax)      #primo campo
movl %val, offset(%eax) #altri campi
```

7. **Pulizia dello stack:** → dopo la chiamata: (ripristiamo lo stack al suo stato originale)

```
addl $4, %esp
```

Abbiamo aggiunto 4 byte allo stack per rimuovere il parametro passato a `malloc` essendo quest'ultimo un parametro a 4 byte (l'intero 8).

Osservazioni importanti

- In assembly non esiste il concetto di tipo: è il programmatore che gestisce gli offset.
- L'allineamento della memoria è importante (tipicamente a multipli di 4 byte).
- Il puntatore restituito deve essere salvato se serve dopo altre operazioni.

Ricordiamo che l'offset è una distanza in byte tra l'inizio della struttura e il campo specifico.

Esempio:

```
int list_add_first(node_t **l, short elem) {
    node_t *p = *l;
    node_t *n = malloc(sizeof(node_t));
    if (n == NULL) return -1;
    n->elem = elem;
    n->next = p;
    *l = n;
    return 0;
}
```

f.c

```
int list_add_first(node_t **l, short elem){
    node_t **si = l;
    node_t *b = *si;
    node_t *a = malloc(sizeof(node_t));
    if (a != NULL) goto F;
    return -1;          // allocation error
F:
    short d = elem;
    (*a).elem = d;
    (*a).next = b;
    *si = a;
    return 0;
}
```

f_eq.c

```
.globl list_add_first
```

```
# typedef struct node_t {    // base
#     short elem;           // offset: 0 |xx..|    (base)
#     struct node_t *next;   // offset: 4 |xxxx|    4(base)
# } node_t;                 // sizeof: 8
```

```
list_add_first:
```

```
    pushl %ebx
    pushl %esi
    subl $4, %esp
```

```
    movl 16(%esp), %esi      # node_t **si = l;
    movl (%esi), %ebx        # node_t *b = *si;
    movl $8, (%esp)
    call malloc              # *a = malloc(sizeof(node_t));
    testl %eax, %eax         # if (a != NULL)
    jnz F                    # goto F;
    movl $1, %eax            # return -1;          // allocation error
    jmp E
```

```
F:
```

```
    movl 20(%esp), %edx      # short d = elem;
    movw %dx, (%eax)         # (*a).elem = d;
    movl %ebx, 4(%eax)       # (*a).next = b;
    movl %eax, (%esi)        # *si = a;
    movl $0, %eax
```

```
E:
```

```
    addl $4, %esp
    popl %esi
    popl %ebx
    ret                      # return 0;
```

f.s

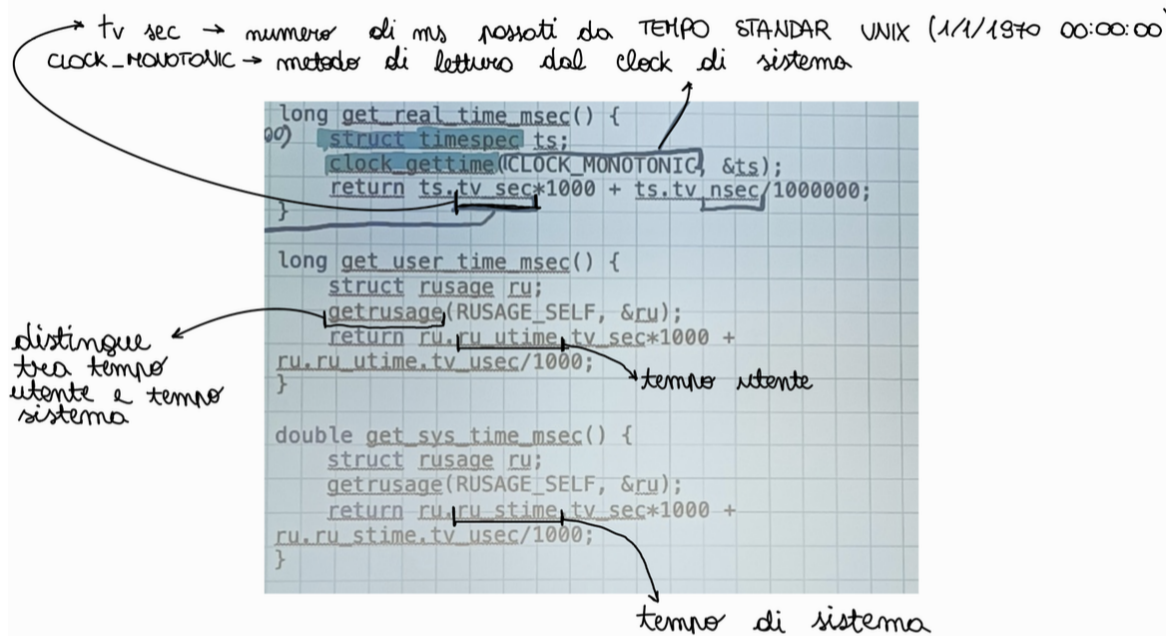
3.15 Profilazione delle prestazioni

L'obiettivo della profilazione è calcolare la latenza di esecuzione di ogni parte di un programma. Si divide in:

- **Manuale** → tramite strumentazione manuale del codice
- **Automatica** → l'strumentazione viene fatta da un tool

Profilazione Manuale

```
main(){
    .
    .
    tempo -> x
    -----
    | target |
    -----
    tempo -> x'
    calcolo x - x' // e sarà il tempo di esecuzione finale
    .
    .
}
```



Quale "tempo" stiamo misurando?



Figura 3.21: Profilazione manuale

Questo tipo di profilazione ha dei limiti:

1. **Latenza dello strumento** → tempo necessario per eseguire la misurazione
2. **Risoluzione dello strumento** → nel nostro caso risulta essere il minimo intervallo temporale misurabile

Importante: la misurazione di un tempo T con uno strumento A è significativa solo se la risoluzione di A è maggiore di T .

Importante: la latenza dello strumento utilizzato per la misura deve influire sui risultati in modo *trascurabile*.

Profilazione Automatica

Per la profilazione automatica ci aiuteremo con il tool **"gprof"**, vediamo come funziona:

1. Bisogna innanzitutto compilare il codice target con **"-pg"**
2. Eseguo il programma compilato
3. Visualizzo i risultati con **"gprof <nome programma>"**

Vediamo un esempio dove c'è un codice che chiama 3 sottoprogrammi: main, A e B.

```
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ gcc -m32 -pg -o e4 mioprogram.c
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ ./e4
0
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ l
total 60K
drwx----- 7 sc sc 224 Apr 14 09:12 .
drwx----- 6 sc sc 192 Feb 20 2024 ..
-rw----- 1 sc sc 18K Apr 17 2024 A_noopt.txt
-rwxrwxr-x 1 sc sc 16K Apr 14 09:12 e4
-rwxrwxr-x 1 sc sc 16K Apr 9 2025 e4unopt
-rw-rw-r- 1 sc sc 2.7K Apr 14 09:12 gmon.out
-rw----- 1 sc sc 470 Feb 20 2024 mioprogram.c
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$
```

Figura 3.22: Esempio del risultato della compilazione

gmon.out è il nostro profilo prestazionale dove all'interno troveremo appunto il tempo impiegato dal nostro programma.

```
sc@scserver:~/shared/training/classes/5.1-profiling/E4-gprof$ gprof e4
Flat profile:

Each sample counts as 0.01 seconds.
%   cumulative   self           calls   self   total    name
time  seconds    seconds             us/call  us/call  name
50.44      6.81      6.81        100000      68.10    68.10  A
49.56     13.50      6.69        100000      66.90    66.90  B
 0.00      13.50      0.00           1         0.00     0.00  init
```

Figura 3.23: gmon.out

C'è inoltre un altro tipo di lista ovvero il **Call graph**, ovvero un grafo che rappresenta le chiamate:

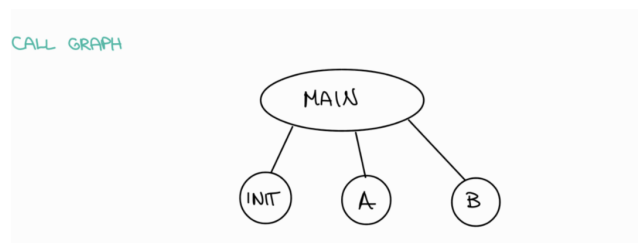


Figura 3.24: Call Graph

Assetti chiave della ottimizzazione

L'obiettivo è ridurre l'uso delle risorse massimizzando le prestazioni (nel nostro caso la latenza) senza impattare la correttezza del programma. In linea generale possiamo intervenire su due livelli:

1. **Algoritmo** → scegliere l'algoritmo con costo computazionale minore che risolve il problema. Tipicamente questa scelta è affidata al progettista, ma in alcuni casi può impattare anche il programmatore. Anche questo ha i suoi pregi e difetti:
 - + La scelta dell'algoritmo ha un impatto molto significativo sulle prestazioni.
 - - Non è sempre possibile
 - - Richiede una implementazione ad-hoc
2. **Implementazione** → ottimizzare il codice che implementa l'algoritmo scelto. Questa parte è affidata tipicamente al programmatore o il compilatore. Anche qui ci sono vantaggi e svantaggi:
 - + Soluzione (parzialmente) automatica
 - + Soluzione "economica" da adottare
 - - Alcune implementazioni possono essere "platform-dependent" (potrebbero funzionare solo su alcune piattaforme)

Tecniche di ottimizzazione del codice

Per l'ottimizzazione del codice abbiamo due approcci principali:

1. **Riduzione del "work"** → ridurre il numero di istruzioni da eseguire.
2. **Riduzione del "costo"** → scelgo tra più alternative quella che impegna la CPU per meno tempo possibile.

Il compilatore applica questi approcci usando numerose tecniche di ottimizzazione attraverso queste opzioni:

- "-O0" → default con cui non applica *NESSUNA* ottimizzazione
- "-O1" → ottimizza
- "-O2" → ottimizza di più
- "-O3" → ottimizza ancora di più
- "-Og" → ottimizza il codice cercando di preservare i simboli di debug
- "-Os" → ottimizza riducendo lo spazio

Abbiamo diverse tecniche di ottimizzazione:

- **Constant Folding** → precalcola i valori costanti presenti nel codice e viene applicata anche ad "-O0":

```
int f(){
    return 8+(14/2)* 3; ---> return 29;
}
```

- **Constant Propagation** → i valori costanti vengono propagati nel codice dove possibile:

```
int f(){
    int x = 8;           (passo 1)      (passo 2)
    return x - 2; ---> return 8-2; ---> return 6;
}
```

- **Common SubExpression Elimination** → sostituisce occorrenze multiple della stessa sottoespressione (un'espressione all'interno del codice che esegue un certo numero di attività):

```
int f(int x, int y){          int z = x+y;    //aggiunge una variabile per semplificare
    return (x+y) * (x+y); ---> return z * z;
}
```

- **Loop Invariant Code Motion** → sposta fuori dal ciclo le espressioni che non dipendono dal ciclo stesso:

<pre>int f(int x, int n){ int s = 1; for(; n > 0; n--) s += 7+x; // ^ sottoespressione che non → // dipende dal ciclo return s; }</pre>	<pre>int f(int x, int n){ int s = 1; int z = 7+x; for(; n > 0; n--) s += z; return s; }</pre>
---	---

ATTENZIONE: non è sempre applicabile. Vediamo un esempio:

```
int g(int k){
    printf("Pippo");
    return k+1;
}
int f(int x, int n){
    int s = 1;
    for( ; n > 0; n--)
        s += g(x); // espressioni come questa o l'uso di variabili globali poichè potrebbero
    return s;      // essere modificate durante il programma a nostra insaputa
}
```

- **Loop Unrolling** → dato un ciclo con un numero di iterazioni noto a tempo di compilazione, riduce o elimina le iterazioni.

<pre>int f(){ int a = 0; for(int i = 0; i < 10; i++) a += 1; return a; }</pre>	<pre>int f(){ int a = 0; a++; . . // ripetuto 10 volte . a++; return a; // ottimizzato con i vari metodi // avremo semplicemente: return 10; }</pre>
---	--

Come facciamo se il numero di iterazioni è noto solo in esecuzione?

<pre>int sum(int *v, int n){ int i,s = 0; for(i = 0; i<n; i++) s += v[i]; // n però è noto solo // a tempo di esecuzione return s; }</pre> Esegue blocchi di dimensione 1 per massimo k - 1 volte	<pre>int sum(int *v, int n){ int i,s = 0; // 3 è il valore k, 2 è k-1 for(i = 0; i + 2 < n; i+=3){ s += v[i]; s += v[i+1]; // eseguo k operazioni s += v[i+2]; // per iterazione } }</pre> Questa esegue blocchi di istruzione di dimensione K per n/k volte (parte intera inferiore della divisione).
---	--

K è un valore arbitrario scelto da noi. Se *n* è *multiplo di k* non avremo problemi, se non è multiplo rischiamo di non eseguire alcune operazioni.

- **Register Allocation** → massimizza l'uso dei registri.

ATTENZIONE: è una ottimizzazione platform dependent

- **Operato Strenght Reduction** → sostituisce una istruzione con un'altra equivalente ma meno costosa.

```
x / 8    ---> x>>3 (shift a destra di 3 bit)
x * 64   ---> x<<6
x * 2    ---> x + x
```

$x * 15 \rightarrow (x \ll 4) - x$

```

void f(int *v, int n, int k){
    int i;
    for(i = 0; i < n; i++){
        v[i] = i * k;
    }
}

void f(int *v, int n, int k){
    int i, c = 0;
    for(i = 0; i < n; i++){
        v[i] = c;
        c += k;
    }
}

```

- **Dead Code Elimination** → elimina il codice non raggiungibile.
- **Riduzione Spazio di Allocazione** → riordinamento dei campi di una struct (dal più grande al più piccolo) per minimizzare il padding (spreco di memoria). Questa va applicata dal programmatore.
- **Allineamento in Memoria** → ogni dato di dimensione S byte viene salvato in memoria ad *indirizzo X* multiplo di S.

Le ottimizzazioni possono essere sia sul codice C (sorgente) che sul codice Assembly, questo varia a seconda del tipo di compilatore che stiamo utilizzando. In generale, le ottimizzazioni a livello di codice sorgente sono più portabili, mentre quelle a livello di Assembly possono essere più specifiche per una piattaforma.

3.16 Come vengono eseguiti i programmi

Quando un programma viene eseguito, diventa un **processo**. In altre parole, un processo è semplicemente un programma in esecuzione. La gestione dei processi è uno dei compiti principali di un sistema operativo. Uno stesso programma può essere avviato più volte, generando così più processi che possono esistere contemporaneamente nel sistema. Ogni processo è identificato in modo univoco da un numero chiamato **PID** (Process ID). Il PID è un valore progressivo che aumenta di uno ogni volta che viene creato un nuovo processo.

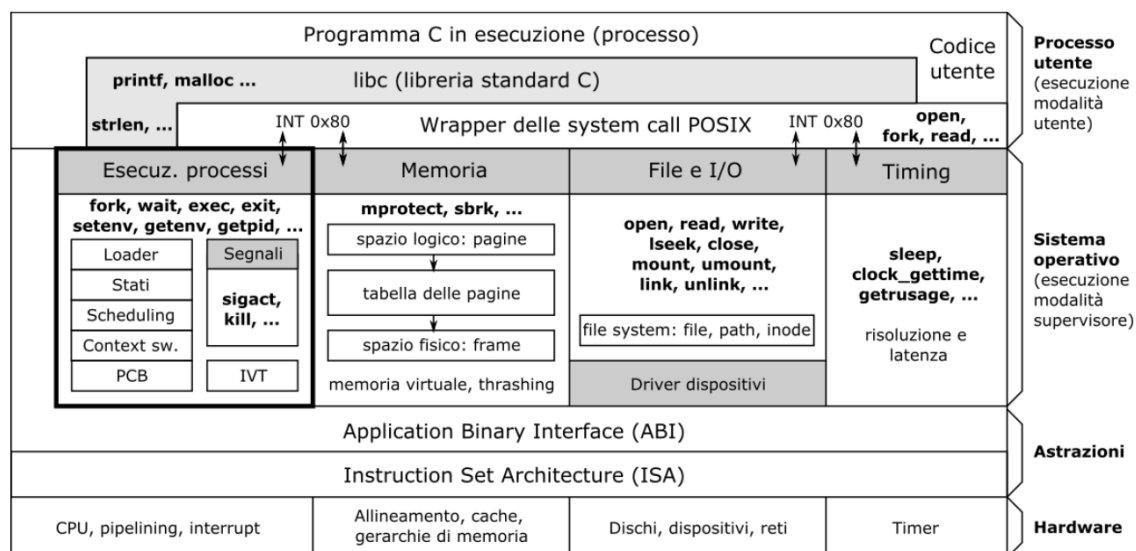


Figura 3.25: Panoramica di un sistema di calcolo Linux/x86

Per la documentazione di posix/libc possiamo utilizzare il comando **"man"**, oppure cercarlo sul web sul sito dell'"open group". "man" prevede diverse sezioni tra cui:

1. *Comandi del terminale (shell)*

Esempio: `man ls`

2. *System call*

Esempio: `man fork`

3. *Funzioni Libc*

Esempio: `man strlen`

Per accedere a una sezione specifica:

Esempio: `man 3 printf`

Terminazione dei programmi

Ogni programma termina restituendo un **codice di terminazione**, detto anche return code, rappresentato da un valore intero. Normalmente per convenzione, il return code è utilizzato per segnalare la corretta terminazione o meno del programma.

- $\emptyset \rightarrow$ "nessun errore"
- Qualsiasi altro valore $\rightarrow$ rappresenta un errore (error code)

Il return code può essere generato in diversi luoghi:

- Se siamo nel `main()`
 - dopo il termine dell'ultima istruzione
 - "return"
- Dopo l'esecuzione di:
 - "exit"
 - "abort"

Gestire la terminazione di un programma

È possibile configurare nel programma una o più di gestione della terminazione, che vengono eseguite quando il programma termina e prendono il nome di **EXIT Handler**.

```
#include <stdlib.h>
int atexit(void (*handler)(void));
```

L'Handler è un gestore di tipo void, quindi un puntatore a funzione che non prende parametri e non restituisce nulla. Esempio:

```
void f(){
    .
    .
    .
}
main(){
    .
    .
    atexit(f);
    .
    .
}
```

Tipologie di Errore

1. **Errori non recuperabili** → causano la terminazione anomala del programma (es. divisione per zero, accesso a memoria non valida, ecc.)
2. **Errori recuperabili** → possono essere gestiti all'interno del programma (es. file non trovato, input non valido, ecc.). Questi errori pongono il programma in uno stato che potrebbe omeno richiedere la sua terminazione (la scelta è del programmatore).

Per restituire il codice di errore spesso le librerie scrivono il codice di errore nella variabile globale **ERRNO**. Ci sono presenti dei **Pattern di Gestione degli Errori**

Figura 3.26: Codice